The Completion Agent

Advanced Machine Learning as Completion

Pasquale Caianiello

Working Draft October 2026
Advanced Machine Learning, Univaq.
Course Textbook

Preface

Machine learning becomes difficult surprisingly quickly. A student may learn nearest-neighbour methods, linear regression, principal components, neural networks, probabilistic models, sequence models, and attention, yet still find that each new method seems to arrive with a new vocabulary and a new set of intuitions. This book is written for graduate students who have already encountered the basic language of machine learning and now want a more connected understanding of why these methods work, how they differ, and what must be checked before their predictions are trusted.

The starting point is deliberately simple. Imagine a data object with one entry missing. The observed entries are available; one quantity is not. The immediate problem is reconstruction:

$$\text{observed part} \quad \longrightarrow \quad \text{plausible value for the missing part.}$$

This is the most literal form of *completion*. It is also a useful first model of learning. To fill the blank we need more than an algorithm: we need some reason why the observed information constrains what is missing. Nearby samples may be relevant, features may obey a linear relation, the data may lie near a low-dimensional subspace, or a representation may have been learned in which the missing quantity becomes easy to infer. Different assumptions lead to different completions.

The book develops advanced machine learning by expanding this simple problem step by step. A missing coordinate becomes a general partial observation. Reconstruction becomes prediction. The relations used for inference become local neighbourhoods, linear subspaces, latent factors, learned representations, probabilistic dependencies, temporal states, and attention patterns. In this sense, *completion* is a pedagogical lens: it asks what information is available, what remains unresolved, and what structure allows the unresolved part to be inferred.

Who this book is for

The intended reader is a graduate student in computer science, engineering, applied mathematics, or a related field. The book assumes working knowledge of linear algebra, multivariable calculus, elementary probability, and the basic supervised-learning setup. Familiarity with Python and numerical experimentation is useful for the companion exercises, but the mathematical development does not depend on a particular software framework.

The level is intentionally intermediate between an introductory machine-learning text and a specialized monograph. Standard results are derived when the derivation clarifies the idea being used later; otherwise the emphasis is on interpretation, assumptions, diagnostics, and

connections between model families. The aim is not to collect every modern architecture. It is to give students a set of reusable questions that remain useful when architectures change.

The organizing questions

Across the book, a small group of questions will recur:

- What is observed, and through what observation mechanism?
- What quantity, relation, or structure remains unresolved?
- What are the relevant objects, and which of them are related?
- What does the chosen representation make easy or difficult to infer?
- How is the unresolved quantity reconstructed, predicted, or otherwise estimated from the available evidence?
- How can we tell whether that estimate is supported, stable, calibrated, or likely to fail?

These questions are deliberately more basic than the names of particular algorithms. A nearest-neighbour method and a neural network may look very different computationally, but both must decide which aspects of the observed data matter for the current prediction. A linear model and a low-rank model both restrict which directions of variation can be represented. A probabilistic model adds not merely a prediction but a description of alternatives and their uncertainty. The common questions make those similarities visible without erasing the differences.

How the book is organized

Part I, *Completion and Geometry*, begins with the concrete reconstruction problem and develops the role of geometry through local, linear, and latent methods. This part establishes the main mathematical vocabulary of the course: observation, representation, relation, projection, residual, and generalization. The goal is to make familiar methods look less like isolated recipes and more like different ways of using structure to constrain an unresolved quantity.

Part II, *Learning the Geometry*, turns from choosing a geometry to learning one. Representation learning and deep networks are introduced as mechanisms for constructing coordinates in which useful relations become easier to express. Probabilistic completion then adds uncertainty: a system may need to represent several plausible answers, not merely return a single point estimate.

Part III, *Structured and Sequential Completion*, asks what changes when the relevant information is distributed across time or across many interacting elements. Recurrent state, sequence models, masking, and attention are treated as ways of controlling which information is available to the current inference.

Part IV, *Regulated Completion*, studies the point at which prediction must interact with ambiguity, dynamics, and commitment. These chapters are more exploratory. They connect standard machine-learning machinery to a broader view of adaptive computation and clearly distinguish established results from proposed research constructs.

How to read and use the book

The chapters are written to support a one-semester graduate course, but they can also be read selectively. The first four chapters form a compact conceptual foundation. Chapters 5–7 provide the core transition to modern representation learning and uncertainty. Chapters 8 and 9 cover sequential and attention-based models. The final part can be used as an advanced module or as a bridge to research projects.

Most chapters contain three complementary kinds of work. First, they develop the mathematical mechanism of the method. Second, they ask what the mechanism means geometrically or informationally. Third, they introduce diagnostics or experiments that can reveal when the attractive interpretation breaks down. Students should therefore not treat the equations, the conceptual discussion, and the experiments as separate layers of the course. The point is to make them constrain one another.

The companion experiments are intentionally small enough to inspect. They are meant to answer questions such as: Did the neighbourhood really change when the representation changed? Which directions did a projection retain? Did a bottleneck learn useful structure or merely copy the input? Is a probability estimate calibrated? Does an attention pattern remain stable under perturbation? Such questions are often more educational than obtaining a slightly better benchmark score.

A note on the completion viewpoint

The word *completion* is used broadly in this book, but the broadening is gradual. It begins with its ordinary meaning: reconstructing a missing part of an object. Later is used to compare regression, classification, latent-variable models, neural representations, probabilistic prediction, and sequence models.

In some chapters this viewpoint provides a common experimental interface; in others it makes a hidden choice of geometry, representation, support, or uncertainty easier to see.

The guiding idea is therefore modest. Many difficult learning problems become easier to understand when we begin with three questions:

What is known?	What is unresolved?	What structure connects the two?

The rest of the book develops increasingly powerful answers.

Contents

Preface i

Notation and reading conventions xx

I Completion and Geometry 1

1 Learning as Completion 2
1.1 The problem before the algorithm . . . 2
1.2 A partially observed object . . . 3
1.2.1 Masking: information is absent . . . 3
1.2.2 Noise: information is present but uncertain . . . 4
1.2.3 Blur and mixing: information is distributed . . . 4
1.2.4 Projection and subsampling: information is compressed . . . 4
1.2.5 Partial observation as unresolved information . . . 4
1.3 What do we mean by geometry? . . . 5
1.3.1 The matrix is not the geometry . . . 6
1.3.2 Completion requires geometry . . . 6
1.4 A geometry may be assumed, constructed, or learned . . . 10
1.4.1 Assumed geometry . . . 10
1.4.2 Geometry constructed from data . . . 11
1.4.3 Learned geometry . . . 12
1.4.4 Why the distinction matters for completion . . . 13
1.5 Supervised learning as completion . . . 14
1.6 Changing the observation changes the problem . . . 14
1.7 Learning the completion rule and using it . . . 16
1.8 Train, test, and generalization . . . 17
1.9 From completion to regulated completion . . . 18
1.10 A common description of a learning problem . . . 18
1.11 One matrix, one hidden entry, four geometries . . . 19
1.11.1 Rows as objects: sample geometry . . . 20
1.11.2 Columns as objects: feature geometry . . . 21
1.11.3 Samples and features as objects: bipartite geometry . . . 22
1.11.4 Projected rows: latent geometry . . . 22
1.11.5 Same hole, different geometries . . . 23
1.12 Where we begin . . . 24

1.13 Questions for Study . . . 24
1.14 References and Further Reading . . . 24

2 Local Completion: Neighbours and Kernels 26
2.1 The first geometric answer . . . 26
2.2 Before locality: global completion baselines . . . 27
2.3 Metrics create neighbourhoods . . . 28
2.3.1 Neighbourhoods . . . 29
2.4 The geometry is not given by the numbers alone . . . 30
2.4.1 Mixed and structured data . . . 30
2.5 The k-nearest-neighbour completion operator . . . 31
2.5.1 Regression . . . 31
2.5.2 Classification . . . 32
2.6 One local rule, several observation patterns . . . 32
2.7 What does k control? Bias, variance, and scale . . . 34
2.7.1 Cross-validation as scale selection . . . 34
2.8 Kernels: from hard neighbours to soft influence . . . 34
2.8.1 Classification with soft support . . . 35
2.8.2 A note on the word kernel . . . 36
2.9 Local support is more than a prediction . . . 36
2.9.1 Neighbourhood radius . . . 36
2.9.2 Class or target disagreement . . . 36
2.9.3 Total kernel support . . . 37
2.10 A worked example . . . 37
2.10.1 Changing the representation changes the induced geometry . . . 38
2.11 Decision regions and Voronoi structure . . . 39
2.12 Why local completion can work . . . 40
2.13 The curse of dimensionality . . . 40
2.13.1 Distance concentration . . . 41
2.13.2 Irrelevant coordinates . . . 41
2.14 When locality fails for reasons other than dimension . . . 41
2.14.1 Nonstationary relevance . . . 41
2.14.2 Different local scales . . . 42
2.14.3 Disconnected but equivalent objects . . . 42
2.14.4 Ambiguous local evidence . . . 42
2.15 Computation and memory . . . 42
2.16 A practical local-completion checklist . . . 43
2.17 Practical experiment: one dataset, three masked tasks . . . 43
2.18 From fixed geometry to fitted geometry . . . 44
2.19 Summary . . . 45

3 Linear Completion: Fitting Global Directions 46
3.1 From neighborhoods to directions . . . 46
3.2 The design matrix and the completion target . . . 47
3.2.1 Multiple outputs . . . 48
3.3 Least squares as geometric completion . . . 48

3.3.1 Projection onto the column space . . . 49
3.3.2 What exactly is being learned? . . . 50
3.4 Feature geometry inside least squares . . . 50
3.4.1 Orthogonal features . . . 51
3.4.2 Correlated features . . . 51
3.5 Conditioning and sensitivity . . . 51
3.6 Ridge regression: regulating the fitted direction . . . 52
3.6.1 Ridge in singular-vector coordinates . . . 52
3.6.2 The constrained form . . . 53
3.7 Why scaling matters again . . . 53
3.8 Bias, variance, and the regularization path . . . 54
3.9 Other parameter geometries: sparsity and the L_1 norm . . . 55
3.10 Linear classification: completing a discrete target . . . 55
3.10.1 Classification is still completion . . . 56
3.11 Logistic completion: from score to probability . . . 56
3.11.1 Why not least squares on class labels? . . . 57
3.12 Multiclass linear completion and softmax . . . 57
3.13 Margins: how far from indecision? . . . 58
3.13.1 Maximum-margin fitting . . . 58
3.14 Loss geometry and what the model treats as an error . . . 59
3.15 A worked regression example . . . 59
3.16 A worked classification example . . . 60
3.17 Prediction diagnostics beyond the fitted value . . . 61
3.17.1 Residual structure . . . 61
3.17.2 The hat matrix: how the fitted values are assembled . . . 62
3.17.3 The hat matrix as a model-induced mask . . . 62
3.17.4 The residual-maker matrix . . . 64
3.17.5 Leverage: the diagonal of the hat matrix . . . 64
3.17.6 Why leverage and residual must be read together . . . 65
3.17.7 Condition number . . . 66
3.17.8 Margin . . . 66
3.18 When global linear completion works . . . 66
3.19 When global linear completion fails . . . 67
3.19.1 Curvature . . . 67
3.19.2 Interactions . . . 67
3.19.3 Regime changes . . . 68
3.19.4 Extrapolation without support . . . 68
3.20 From raw coordinates to basis expansions . . . 68
3.21 Linear completion under arbitrary masks . . . 69
3.22 The linear completion protocol . . . 69
3.23 Companion experiment: local versus linear completion . . . 70
3.24 A minimal software interface . . . 72
3.25 From fitted directions to latent directions . . . 72
3.26 Summary . . . 73

4 Latent Completion: PCA, SVD, and Low Rank **75**
4.1 From fitted directions to latent structure . . . 75
4.2 Why reduce dimension? . . . 76
4.3 Clustering as a discrete latent representation . . . 77
4.3.1 The k-means objective . . . 77
4.3.2 The geometry determines what a cluster can be . . . 78
4.3.3 Hierarchies and soft memberships . . . 80
4.3.4 Clustering as a bridge from local to latent structure . . . 80
4.4 Centering, covariance, and the shape of a cloud . . . 80
4.4.1 Variance is a geometric criterion, not a universal notion of importance . . . 82
4.4.2 Scaling changes the representation and the induced geometry . . . 82
4.5 PCA as encoding and reconstruction . . . 82
4.5.1 PCA projection as a latent mask . . . 83
4.5.2 PCA reconstruction is not entrywise missing-data completion . . . 84
4.6 Explained variance and selecting the latent dimension . . . 85
4.7 Singular-value decomposition . . . 86
4.7.1 Outer-product decomposition . . . 86
4.8 Why truncated SVD is the best low-rank reconstruction . . . 87
4.8.1 A spectral filter . . . 87
4.8.2 From the SVD to a completion operator . . . 88
4.9 Matrix factorization: latent representations for two object types . . . 93
4.9.1 Non-uniqueness of the factors . . . 94
4.10 From low-rank approximation to matrix completion . . . 95
4.10.1 Factorized formulation . . . 96
4.11 Nuclear-norm regularization . . . 96
4.12 Why missing entries are not automatically recoverable . . . 96
4.13 Alternating minimization: fitting coupled latent factors . . . 97
4.14 Latent representation for a new sample . . . 97
4.15 PCA with missing values: an important distinction . . . 98
4.16 Denoising as reconstruction from a corrupted observation . . . 99
4.17 Reconstruction error as a diagnostic . . . 99
4.17.1 Residual reconstruction . . . 99
4.17.2 Spectral decay . . . 100
4.17.3 Observation-conditioned reconstruction error . . . 100
4.17.4 Support for latent inference . . . 100
4.18 When low rank is the wrong structural assumption . . . 101
4.18.1 Curved manifolds . . . 101
4.18.2 Multiple regimes . . . 101
4.18.3 Sparse but high-rank structure . . . 101
4.18.4 Rare directions can matter . . . 101
4.18.5 Biased or structured observation processes . . . 101
4.18.6 Scale, observation model, and loss mismatch . . . 101
4.19 Learning a latent representation and using it . . . 102
4.20 Student experiment: one matrix, several latent models . . . 102
4.21 A minimal software interface . . . 104
4.22 From linear latent representation to learned representation . . . 104

4.23 Summary . . . 105
4.24 Exercises . . . 107
4.25 References and Further Reading . . . 107

II Learning the Geometry 109

5 Representations and Autoencoders 111
5.1 From latent coordinates to learned maps . . . 111
5.2 What is a representation? . . . 112
5.2.1 Representation is not merely compression . . . 114
5.3 Encoder–decoder completion . . . 114
5.3.1 The autoencoder as a completion operator . . . 115
5.3.2 From projection masks to learned information selection . . . 115
5.4 The linear autoencoder and PCA . . . 116
5.5 Nonlinear autoencoders . . . 117
5.6 The bottleneck and the identity problem . . . 118
5.7 Denoising: completion made explicit . . . 118
5.8 Masked autoencoding . . . 119
5.8.1 Why supply the mask? . . . 120
5.8.2 Random and structured masks . . . 120
5.8.3 External mask and internal information selection . . . 120
5.9 What does the representation learn to preserve? . . . 121
5.9.1 Invariant structure . . . 121
5.9.2 Equivariant structure . . . 121
5.9.3 Discriminative structure . . . 122
5.10 Latent geometry in mathematical form . . . 122
5.10.1 The decoder has geometry too . . . 123
5.11 Interpolation in representation space . . . 124
5.12 Regularizing the representation . . . 124
5.12.1 Sparse representations . . . 124
5.12.2 Contractive representations . . . 124
5.12.3 Weight regularization . . . 125
5.13 Reconstruction quality is not representation quality . . . 125
5.13.1 A linear probe . . . 125
5.14 The representation depends on the mask . . . 126
5.15 Missingness is not always neutral . . . 126
5.16 Failure modes of autoencoding geometry . . . 126
5.16.1 Identity or memorization . . . 127
5.16.2 Bottleneck too narrow . . . 127
5.16.3 Decoder dominance . . . 127
5.16.4 Wrong corruption model . . . 127
5.16.5 Latent folding . . . 127
5.16.6 Holes in latent space . . . 127
5.16.7 Scale and loss mismatch . . . 127
5.16.8 Shortcut geometry . . . 127

5.17 Diagnostics for learned representations and induced geometry . . . 128
5.17.1 Held-out reconstruction . . . 128
5.17.2 Mask-conditioned error . . . 128
5.17.3 Neighborhood stability . . . 128
5.17.4 Local sensitivity . . . 128
5.17.5 Linear accessibility . . . 128
5.17.6 Interpolation and perturbation tests . . . 129
5.17.7 Train–test latent support . . . 129
5.18 From reconstruction to self-supervision . . . 129
5.19 Learning and using an information-selective representation . . . 129
5.20 Companion experiment: learning an information-selective representation . . . 130
5.21 A minimal software interface . . . 132
5.22 From learned representation to deep completion . . . 132
5.23 Summary . . . 133

6 Deep Completion 135
6.1 Why neural networks enter naturally here . . . 135
6.2 From one fitted direction to a layer . . . 136
6.3 A layer as a discrete activation field . . . 137
6.4 The multilayer perceptron as successive completion geometry . . . 138
6.5 Nonlinearity: bending a global geometry . . . 138
6.5.1 ReLU as a state-dependent hard mask . . . 139
6.5.2 Local information filtering and induced geometry . . . 139
6.6 Expressivity is not the same as learning . . . 140
6.7 Outputs are completion heads . . . 141
6.7.1 Regression . . . 141
6.7.2 Binary classification . . . 141
6.7.3 Multiclass classification . . . 141
6.7.4 Reconstruction and masked completion . . . 141
6.8 Training the distributed transformation . . . 142
6.8.1 Gradient descent . . . 142
6.9 Backpropagation: local credit through a composition . . . 142
6.10 Depth and the conditioning of information flow . . . 143
6.10.1 Initialization . . . 144
6.10.2 Normalization . . . 144
6.11 Residual networks: depth as a discrete flow . . . 144
6.12 Convolution: locality built into the coupling . . . 146
6.13 Graph neural networks: geometry made explicit . . . 146
6.14 Regularization: constraining which geometry is learned . . . 148
6.14.1 Weight decay . . . 148
6.14.2 Dropout . . . 148
6.14.3 Data augmentation . . . 148
6.14.4 Early stopping . . . 148
6.14.5 Corruption and masking . . . 148
6.15 Overparameterization and implicit bias . . . 149
6.16 Completion under masks in a deep network . . . 149

6.17 What should we inspect inside a deep network? . . . 150
6.17.1 Layerwise linear accessibility . . . 150
6.17.2 Neighborhood structure . . . 150
6.17.3 Perturbation stability . . . 150
6.17.4 Jacobian gains . . . 151
6.17.5 Activation-mask statistics . . . 151
6.17.6 Gradient statistics . . . 151
6.18 Failure modes . . . 151
6.18.1 Optimization failure . . . 151
6.18.2 Generalization failure . . . 151
6.18.3 Shortcut geometry . . . 151
6.18.4 Representation collapse . . . 152
6.18.5 Saturation or dead units . . . 152
6.18.6 Brittleness . . . 152
6.18.7 Unjustified confidence . . . 152
6.19 A companion experiment: watch geometry form through depth . . . 152
6.20 A minimal software interface . . . 154
6.21 From neural networks to fields . . . 155
6.22 From deterministic completion to uncertainty . . . 155
6.23 Summary . . . 156
6.24 Exercises . . . 157
6.25 References and Further Reading . . . 157

7 Probabilistic Completion and Uncertainty 159
7.1 From one completion to many . . . 159
7.2 From information masks to uncertainty . . . 160
7.2.1 The exact Gaussian case: the complementary mask is the uncertainty covariance . . . 161
7.2.2 The regression hat matrix splits uncertainty as well as information . . . 161
7.2.3 Noisy observations and correlated uncertainty . . . 162
7.3 Conditional distributions as completion operators . . . 163
7.3.1 The information function belongs in the condition . . . 164
7.4 Likelihood as a completion criterion . . . 164
7.4.1 Squared error as Gaussian likelihood . . . 164
7.4.2 Binary cross-entropy as Bernoulli likelihood . . . 165
7.4.3 Softmax cross-entropy as categorical likelihood . . . 165
7.5 Proper scoring and truthful uncertainty . . . 165
7.6 Learning the width of a completion . . . 166
7.7 Probability as a landscape over completions . . . 167
7.8 Bayesian completion . . . 168
7.9 Aleatoric and epistemic uncertainty . . . 169
7.10 Classification uncertainty . . . 170
7.11 Calibration . . . 170
7.11.1 Temperature scaling . . . 171
7.12 Regression intervals: coverage and sharpness . . . 171
7.13 Out-of-distribution inputs and geometric support . . . 172

7.14 Multimodal completion . . . 172
7.15 Latent-variable probabilistic completion . . . 173
7.16 Masked probabilistic completion . . . 174
7.16.1 Consistency on observed coordinates . . . 174
7.17 Diffusion models: completion by repeated denoising . . . 175
7.18 From uncertainty to decision . . . 175
7.18.1 The reject option . . . 177
7.19 Selective prediction and coverage . . . 177
7.20 Neural networks as probabilistic field generators . . . 178
7.21 Diagnostics for probabilistic completion . . . 178
7.21.1 Predictive quality . . . 178
7.21.2 Uncertainty structure . . . 179
7.21.3 Information residual structure . . . 179
7.21.4 Geometric support . . . 179
7.22 A probabilistic Completion Agent interface . . . 180
7.23 Companion experiment: where uncertainty lives and when the width matters . 181
7.23.1 Exact projection baseline . . . 181
7.23.2 Synthetic heteroscedastic regression . . . 181
7.23.3 A deliberate support gap . . . 182
7.23.4 Classification extension . . . 182
7.24 Failure modes . . . 183
7.24.1 Likelihood misspecification . . . 183
7.24.2 Variance as an escape route . . . 183
7.24.3 Overconfident neural probabilities . . . 183
7.24.4 Poor epistemic approximation . . . 183
7.24.5 Calibration drift . . . 183
7.24.6 Averaging incompatible alternatives . . . 183
7.24.7 Confusing the mask with the distribution . . . 184
7.24.8 Uncertainty without action . . . 184
7.25 From distributions to sequences . . . 184
7.26 Summary . . . 185
7.27 Exercises . . . 186
7.28 References and Further Reading . . . 187

III Structured and Sequential Completion 188

8 Sequential Completion 190
8.1 When partial observation acquires direction . . . 190
8.2 A sequence has more than one geometry . . . 191
8.3 State: compressing the usable past . . . 192
8.4 The Markov idea: when the present is enough . . . 192
8.4.1 Window models . . . 193
8.5 Latent state and state-space models . . . 193
8.5.1 Filtering as repeated completion . . . 193
8.6 Recurrent neural networks: a learned state update . . . 194

8.7 The recurrent state as a persistent activation field . . . 194
8.8 Unrolling recurrence . . . 195
8.9 Backpropagation through time . . . 195
8.10 Gating: learning what persists . . . 196
8.11 Memory is selective, not archival . . . 197
8.12 Teacher forcing and the geometry of training . . . 198
8.13 Forecasting as block completion . . . 198
8.13.1 Recursive one-step completion . . . 199
8.13.2 Direct horizon-specific completion . . . 199
8.13.3 Joint block completion . . . 199
8.14 Sequence classification: completion after compression . . . 199
8.15 Bidirectional recurrence and smoothing . . . 200
8.16 State-space models revisited: learned dynamics . . . 200
8.16.1 Stability and memory . . . 201
8.17 Temporal receptive fields . . . 201
8.18 The usable present . . . 201
8.19 Diagnostics for sequential completion . . . 204
8.19.1 Horizon-wise error . . . 204
8.19.2 State sensitivity to past perturbations . . . 204
8.19.3 Completion sensitivity . . . 204
8.19.4 State norm and saturation . . . 205
8.19.5 State separability . . . 205
8.19.6 Free-run versus teacher-forced performance . . . 205
8.20 A companion experiment: discovering a memory horizon . . . 205
8.21 A minimal sequential Completion Agent interface . . . 206
8.22 Failure modes . . . 207
8.22.1 Too little state . . . 207
8.22.2 Too much undisciplined state . . . 207
8.22.3 Vanishing influence . . . 207
8.22.4 Exploding dynamics . . . 207
8.22.5 Teacher-forcing mismatch . . . 207
8.22.6 Temporal leakage . . . 207
8.22.7 One-state bottleneck . . . 207
8.23 From recurrence to attention . . . 208
8.24 Summary . . . 208
8.25 Exercises . . . 209
8.26 References and Further Reading . . . 209

9 Attention and Masked Completion 210
9.1 From persistent state to addressable context . . . 210
9.2 Attention as a learned relation . . . 211
9.2.1 A graph view . . . 212
9.3 Queries, keys, and values . . . 212
9.4 Why the scale factor appears . . . 214
9.5 Softmax as selective coupling . . . 214
9.6 The usable present becomes dynamic . . . 215

9.7 Self-attention . . . 215
9.8 Multi-head attention: several geometries at once . . . 216
9.9 Attention masks: geometry by permission . . . 217
9.9.1 Causal mask . . . 217
9.9.2 Bidirectional visibility . . . 217
9.9.3 Local and structured masks . . . 218
9.10 Position must be represented . . . 218
9.11 A transformer block as repeated geometric reorganization . . . 218
9.12 Masked-token completion . . . 220
9.12.1 Why masking learns representations . . . 221
9.13 Autoregressive completion . . . 221
9.14 Masked and autoregressive objectives are different observation problems . . . 222
9.15 Cross-attention: completion between two objects . . . 222
9.16 Transformer families through the completion lens . . . 223
9.16.1 Bidirectional encoders . . . 223
9.16.2 Causal decoders . . . 223
9.16.3 Encoder-decoder systems . . . 223
9.17 Attention is not a complete theory of relevance . . . 223
9.18 Diagnostics for attention geometry . . . 224
9.18.1 Entropy and concentration . . . 224
9.18.2 Distance profile . . . 224
9.18.3 Head diversity . . . 224
9.18.4 Mask compliance . . . 225
9.18.5 Perturbation consistency . . . 225
9.18.6 Usable-present profile . . . 225
9.19 Failure modes . . . 225
9.19.1 Attention without useful representation . . . 225
9.19.2 Diffuse attention without discrimination . . . 225
9.19.3 Prematurely sharp attention . . . 225
9.19.4 Positional shortcuts . . . 225
9.19.5 Context dilution . . . 226
9.19.6 Autoregressive drift . . . 226
9.19.7 Quadratic resource cost . . . 226
9.20 A companion experiment: associative retrieval under distractors . . . 226
9.20.1 Data construction . . . 226
9.20.2 Models . . . 226
9.20.3 Measurements . . . 227
9.20.4 Expected interpretation . . . 227
9.21 A second experiment: change only the mask . . . 227
9.22 Attention and completion fields . . . 228
9.23 Toward regulation . . . 229
9.24 Chapter summary . . . 229
9.25 Questions for study . . . 230
9.26 References and Further Reading . . . 230

IV Regulated Completion **232**

10 Ambiguity, Shift, and Abstention **234**
10.1 From prediction to commitment . . . 234
10.2 When the observation does not determine one answer . . . 235
10.2.1 Outcome ambiguity . . . 235
10.2.2 Knowledge uncertainty . . . 235
10.2.3 Weak geometric support . . . 235
10.2.4 Structural ambiguity . . . 236
10.3 Confidence is not support . . . 236
10.4 Distribution shift . . . 237
10.4.1 Covariate shift . . . 237
10.4.2 Label shift . . . 238
10.4.3 Conditional or concept shift . . . 238
10.4.4 Why the taxonomy matters . . . 238
10.5 Out-of-distribution inputs . . . 238
10.5.1 Distance and density diagnostics . . . 239
10.5.2 Predictive and energy scores . . . 239
10.5.3 Ensemble disagreement . . . 239
10.5.4 Typicality is not likelihood alone . . . 239
10.6 Calibration under shift . . . 240
10.7 Selective prediction . . . 240
10.8 The reject option as a decision problem . . . 242
10.9 Set-valued completion . . . 243
10.10 Conformal prediction: uncertainty with a coverage guarantee . . . 243
10.10.1 Calibration scores . . . 243
10.10.2 What the guarantee does and does not say . . . 244
10.10.3 Set size as a regulatory readout . . . 244
10.11 Binary decisions with an unresolved region . . . 245
10.12 Why unresolved is more than “low confidence” . . . 245
10.13 A completion license . . . 246
10.14 Beyond abstention: preserve, ask, retrieve, defer . . . 247
10.15 A field view of commitment . . . 248
10.16 Companion experiment: confidence, support, and shift . . . 248
10.16.1 Data . . . 248
10.16.2 Models . . . 249
10.16.3 Readouts . . . 249
10.16.4 Selective evaluation . . . 249
10.16.5 Ternary map . . . 250
10.17 A second experiment: what conformal coverage survives . . . 250
10.18 From static regulation to completion dynamics . . . 251
10.19 Chapter summary . . . 251
10.20 Questions for study . . . 252
10.21 References and Further Reading . . . 253

11 Completion as Dynamics **254**
11.1 From static regulation to state evolution . . . 254
11.2 Direct maps versus relaxation . . . 255
11.3 Clamped variables and free variables . . . 256
11.3.1 Soft clamping . . . 257
11.4 A state field over a geometry . . . 257
11.5 Propagation through a geometry . . . 258
11.5.1 The corresponding energy . . . 259
11.5.2 Beyond averaging . . . 260
11.6 Fixed points and equilibrium completion . . . 260
11.6.1 Contraction and local stability . . . 260
11.6.2 Continuous-time relaxation . . . 262
11.7 Energy, Lyapunov functions, and attractors . . . 263
11.8 Associative memory as completion by relaxation . . . 263
11.9 Residuals as dynamical readouts . . . 264
11.9.1 Observed inconsistency . . . 264
11.9.2 Dynamic residual . . . 264
11.9.3 Constraint residual . . . 265
11.9.4 Competition and perturbation residuals . . . 265
11.10 Regulation changes the dynamics . . . 265
11.10.1 Regulation versus learning . . . 266
11.11 When the effective relation geometry becomes state-dependent . . . 267
11.12 Ambiguity as multistability and slow resolution . . . 267
11.12.1 Multiple attractors . . . 267
11.12.2 Slow relaxation . . . 268
11.12.3 Oscillation is not uncertainty . . . 268
11.13 From one relaxation field to coupled fields . . . 268
11.13.1 Why use more than one field? . . . 270
11.14 A minimal regulated completion loop . . . 271
11.15 A dynamical completion license . . . 272
11.16 Companion experiment I: associative pattern completion . . . 272
11.16.1 Patterns . . . 272
11.16.2 Masks and cues . . . 272
11.16.3 Record the trajectory . . . 273
11.16.4 Regulator-on versus regulator-off . . . 273
11.17 Companion experiment II: harmonic completion on a graph . . . 273
11.18 What can go wrong in a relaxation machine . . . 274
11.18.1 Premature locking . . . 274
11.18.2 Dead relaxation . . . 274
11.18.3 Oscillation and instability . . . 274
11.18.4 Spurious equilibria . . . 274
11.18.5 Over-smoothing . . . 274
11.18.6 Path dependence . . . 275
11.18.7 Regulator-induced pathology . . . 275
11.19 What Chapter 11 adds to the completion framework . . . 275
11.20 Chapter summary . . . 276

11.21 Questions for study . . . 276
11.22 References and Further Reading . . . 277

12 The Completion Agent **278**
12.1 From a completion rule to a completion machine . . . 278
12.2 The anatomy of one completion episode . . . 279
12.3 Evidence does not merely enter the machine: it constrains it . . . 281
12.4 The completion field . . . 282
12.5 The regulatory field . . . 283
12.6 Geometry becomes an active variable . . . 284
12.7 Continuous relaxation as computation . . . 284
12.7.1 Speed of resolution . . . 285
12.7.2 Trajectory shape . . . 285
12.7.3 Reopening . . . 285
12.7.4 Persistent unresolvedness . . . 285
12.7.5 Multiple time scales . . . 285
12.8 A concrete coupled-field realization . . . 285
12.9 Why a second field changes what can be represented . . . 286
12.10 The usable present becomes a controlled dynamical object . . . 287
12.11 From completion to action . . . 288
12.12 Information-seeking actions . . . 288
12.13 Learning across completion episodes . . . 289
12.14 The Completion Agent protocol . . . 289
12.15 Classical learners inside the protocol . . . 290
12.16 A reference Completion Agent . . . 290
12.17 What should be measured . . . 292
12.17.1 Completion quality . . . 292
12.17.2 Support and diagnostic quality . . . 293
12.17.3 Regulatory quality . . . 293
12.17.4 Trajectory quality . . . 293
12.18 A decisive experiment . . . 293
12.19 From software to physical completion . . . 294
12.20 What the book has changed . . . 295
12.21 Chapter summary . . . 295
12.22 Questions for study . . . 296
12.23 References and Further Reading . . . 297

A Linear Algebra and Matrix Facts Used in This Book **298**
A.1 Vectors, matrices, and shapes . . . 298
A.1.1 Transpose, inner products, and outer products . . . 299
A.1.2 Hadamard product . . . 299
A.2 Subspaces, span, rank, and null spaces . . . 299
A.3 Norms and matrix size . . . 300
A.4 Orthogonality and orthonormal bases . . . 300
A.5 Special matrix classes . . . 301
A.5.1 Diagonal matrices . . . 301

A.5.2 Symmetric matrices . . . 301
A.5.3 Idempotent matrices and projectors . . . 301
A.5.4 Positive semidefinite matrices . . . 302
A.6 Trace, quadratic forms, and useful identities . . . 302
A.7 Linear systems, inverses, and conditioning . . . 303
A.8 Least squares and orthogonal projection . . . 303
A.8.1 Leverage . . . 304
A.9 QR factorization . . . 304
A.10 Eigenvalues, eigenvectors, and the spectral theorem . . . 304
A.11 Singular-value decomposition . . . 305
A.11.1 Low-rank approximation . . . 305
A.12 Pseudoinverse . . . 305
A.13 Ridge regression as spectral filtering . . . 306
A.14 Covariance as a matrix . . . 306
A.15 PCA in one page . . . 307
A.16 Jacobians and local linearization . . . 307
A.17 Diagonal gates and ReLU masks . . . 308
A.18 Graph matrices . . . 308
A.19 Attention matrices . . . 308
A.20 A compact dictionary of recurring operators . . . 309
A.21 Useful identities at a glance . . . 309
A.22 What to recognize when reading the main text . . . 310

Epilogue: From Completion Protocols to Research Machines **311**

References **333**

Glossary **339**

List of Figures

1.1 Whole-matrix completion in matrix space. The unit of reasoning is a complete matrix object: a partial observation constrains the admissible completions, while a geometry over whole matrices supplies relational support. 7

1.2 Changing the observation operator changes the matrix-completion problem. Random masking, structured row or column occlusion, corruption, and coarse observation leave different information unresolved and therefore induce different completion tasks. 15

1.3 The same hidden entry x_{21} completed under four different geometries. The data and the missing quantity are fixed; only the objects, relations, and completion mechanism change. 20

4.1 The same samples under different clustering geometries. 79

5.1 The same represented matrices can support different geometries. Similarity, directional or transition, and learned transformation relations organize the same matrix objects differently; representation alone does not determine the relation structure. 113

6.1 A deep network as successive transformations of a distributed activation field. Each layer changes both the state and the local geometry through which later completion becomes simple. 145

6.2 Message passing on an explicit relational geometry. A masked node state is completed from a neighbourhood whose connectivity is supplied by the graph and whose messages are learned. 147

7.1 Diffusion as probabilistic completion by repeated denoising. A forward corruption process destroys structure; learned reverse transitions reconstruct it step by step. 176

8.1 History, stored state, structural receptive field, and usable present are distinct. Only part of accumulated history may exert effective influence on the current completion. 202

9.1 Attention dynamically constructs a relation field and therefore a query-dependent usable present. Different heads can expose different relational geometries over the same represented positions. 219

10.1 Confidence, geometric support, and ambiguity are different axes. A selective policy should not collapse them into one scalar unless the task justifies doing so. 241

11.1 Completion as a trajectory rather than a one-shot map. Different masked initial states relax through the same coupling geometry toward stable, metastable, or unresolved regimes. 261
11.2 From one activation field to coupled completion and regulatory fields. The completion state shapes regulatory readouts, while regulation modulates gain, coupling, scale, or stopping during relaxation. 269

12.1 A tri-field refolding with evidence, completion, and regulatory fields, together with an action readout. Different time scales separate fast completion relaxation, intermediate regulation, and slower persistent learning. 280
12.2 The Completion Agent protocol. An observation rule supplies evidence; when appropriate a coordinate mask determines a clamped state; information propagates through an induced geometry; a completion is exposed with diagnostics; regulation selects commitment, preservation, information-seeking action, deferral, or refusal. 291

A.1 From token completion to a regulated language research machine. Attention builds a dynamic usable present, retrieval can extend it, and a regulator chooses among continuation, retrieval, clarification, abstention, and action. 316
A.2 Medical-image diagnosis as an evidence chain rather than an image-to-label arrow. Image findings, clinical context, uncertainty, and support are completed separately before a diagnostic commitment or referral action. 321
A.3 Genomic interpretation as multiscale completion across sequence, regulatory context, gene expression, pathways, phenotype, and clinical evidence. Strong support at one link need not license the final interpretation. 325

Notation and reading conventions

This book uses a deliberately small set of recurring symbols. The notation is not meant to imply that every learning system has the same internal architecture; it is a common descriptive language for comparing how systems use incomplete information.

Symbol	Typical meaning
X	A data object, often a matrix of samples by features.
M	A coordinate observation mask when the observation is literal entry-wise visibility.
ω	A general observation specification or information-access rule.
$\mathcal{O}_\omega(X)$	The observation of X supplied under specification ω; masking is one special case.
$\widehat{X}$	A proposed completion or reconstruction.
$\mathcal{G} = (\mathcal{V}, \mathcal{R}, W)$	A geometry: objects, admissible relations, and a relation descriptor or strength.
z, h	Latent or internal state variables.
W, A	Coupling or relation matrices; the interpretation depends on context.
$\mathcal{C}$	A completion operator.
$\mathcal{L}$	A loss, objective, or energy used to fit or relax a model.
$\mathcal{P}_t$	The usable present at time t: information from accumulated history that is effectively available to the current completion.
R	Regulatory state or regulatory readout when the distinction is useful.

Three kinds of statement

The book intentionally mixes three levels of discourse. Standard results from machine learning and statistics are presented with references. The *completion interpretation* is the organizing viewpoint of this book: it often repackages standard methods in a common language, and the text states explicitly when the gain is mainly conceptual. Finally, the later chapters introduce a research program—usable presents, completion licenses, and coupled regulatory fields—whose stronger claims are proposals to be tested rather than established facts about existing systems.

Throughout, the phrase "geometry" is used operationally. It may be induced by a metric, covariance structure, graph, kernel, learned representation together with a relation rule, or

a state-dependent relation field. A representation and a geometry are not identical objects: the representation supplies coordinates or state, while relations defined on that representation induce the geometry. The relevant question is always the same: which objects can influence which others, and through what relation?

Part I

Completion and Geometry

Chapter 1

Learning as Completion

1.1 The problem before the algorithm

Machine learning is usually introduced through a taxonomy of tasks. In supervised learning, inputs are mapped to labels or numerical targets. In unsupervised learning, structure is extracted from unlabelled data—for example clusters, latent factors, or low-dimensional organization. In representation learning, useful internal variables are constructed. In generative modelling, new observations are produced from a learned distribution. Each task then brings its own algorithms, terminology, losses, and evaluation procedures.

This organization is historically natural, but conceptually unsatisfactory. It makes machine learning appear as a collection of techniques before identifying the common problem those techniques address.

We begin instead from a simpler situation.

A learner is presented with an object that is only partly available. Some information is present; some information is absent, hidden, corrupted, unresolved, or yet to be produced. The learner must use the available part to determine something about the unavailable part.

We call this operation completion.

The word is intended broadly. Completion may mean estimating a missing numerical entry, assigning a class, recovering a corrupted image, predicting a future observation, reconstructing a latent variable, generating a continuation, or deciding that the available information does not yet justify a unique answer.

The basic situation is therefore

$$\text{partial observation} \quad \longrightarrow \quad \text{completion.}$$

The central proposal of this book is that a large part of machine learning can be organized around this operation.

This is not the claim that all learning algorithms are identical, nor that every learning problem is literally a missing-data problem. The claim is structural.

Principle 1.1 (Completion viewpoint) *Machine-learning methods differ largely in what they regard as the partially observed object, in the geometry they impose on that object, and in the rule by which information is propagated from the observed part to the unobserved part.*

This viewpoint will allow apparently different methods to be compared within a common language.

> **Scope of the claim.** The completion vocabulary is not useful merely because any prediction can be redescribed as "filling something in." It earns its place only when it makes a modelling choice explicit—for example the observation mask, the geometry of influence, the preservation of several admissible completions, or the decision to remain unresolved. Where none of these distinctions matters, standard task-specific language is usually clearer.

1.2 A partially observed object

Let X denote an object about which we want to make an inference. Depending on the problem, X may be a feature vector, a data matrix, an image, a sequence, a graph, or some other structured object.

A learning machine does not necessarily receive X directly. Instead it receives an *observation* of X,

$$Y = \mathcal{O}(X),$$

where $\mathcal{O}$ is an observation operator.

We shall call X *partially observed* whenever the available observation Y does not determine X completely or reliably.

Here "partial" should therefore be understood in an informational sense. It does not mean only that some coordinates are absent. Information about X may be missing, corrupted, mixed, compressed, or otherwise transformed.

1.2.1 Masking: information is absent

The simplest case is coordinate masking. Let

$$M \in \{0,1\}^{n \times d}$$

be a binary mask. The observation operator is

$$\mathcal{O}_M(X) = M \odot X,$$

where $M_{ij} = 1$ means that x_{ij} is observed and $M_{ij} = 0$ means that it is not.

For example,

$$X = (x_1, x_2, x_3), \qquad M = (1, 1, 0)$$

produces the partial observation

$$Y = (x_1, x_2, ?).$$

The open component is literal: x_3 has not been observed.

This case will be particularly useful throughout the book because it makes the completion problem visible:

$$(Y, M) \longrightarrow \widehat{X}.$$

But masking is only one way in which an object may be partially observed.

1.2.2 Noise: information is present but uncertain

Suppose instead that every coordinate is measured, but measurements are corrupted:

$$Y = X + \varepsilon.$$

Now there is no missing coordinate and a binary mask would simply be

$$M = \mathbf{1}.$$

Nevertheless, X is not completely known. We observe Y, not the clean object X. Denoising asks for an estimate

$$Y \longrightarrow \widehat{X}.$$

The unresolved information is therefore not *where* a value is missing, but *which part of the observed value belongs to the object and which part belongs to the corruption.*

1.2.3 Blur and mixing: information is distributed

An image may be observed through a blur operator K:

$$Y = KX + \varepsilon.$$

Here each observed value may contain information from several locations of the original image. The information has not simply disappeared; it has been mixed.

Recovering X requires reversing, approximately, the effect of the observation process:

$$KX + \varepsilon \longrightarrow \widehat{X}.$$

1.2.4 Projection and subsampling: information is compressed

Another possibility is

$$Y = AX,$$

where A maps the original object into a lower-dimensional observation.

Downsampling an image, taking a limited collection of measurements, or observing a projection of a high-dimensional object all have this form.

Again, no binary mask on the coordinates of Y expresses the whole problem. The important fact is that many possible objects X may be compatible with the same observation Y.

1.2.5 Partial observation as unresolved information

These examples have different observation mechanisms:

$Y = M \odot X$	:	information is absent,
$Y = X + \varepsilon$	:	information is corrupted,
$Y = KX + \varepsilon$	:	information is mixed,
$Y = AX$	:	information is compressed.

What they have in common is not a particular mask. It is that the observation leaves some aspect of the underlying object unresolved.

This motivates the broader completion problem

$$Y = \mathcal{O}(X) \qquad \longrightarrow \qquad \widehat{X},$$

or, when only part of X is of interest,

$$Y \qquad \longrightarrow \qquad \widehat{T(X)},$$

where $T(X)$ denotes the component, label, latent quantity, or future value that we want to infer.

A mask is therefore a particularly simple and important *observation operator*, not the universal form of partial observation.

Principle 1.2 (Partial observation) *An object is partially observed whenever the information currently available does not determine the quantity of interest completely or reliably. Missing values are one case; noise, blur, subsampling, and projection are others.*

1.3 What do we mean by geometry?

The term *geometry* will be used throughout this book in an operational sense.

Definition 1.3 (Geometry) *geometry is the structure that specifies how the objects in a representation are related.*

A geometry answers three questions:

What are the objects?
Which objects are related?
What does this relation do or encode?

The relation may be represented by a distance, similarity, inner product, covariance, kernel, graph edge, transition probability, attention weight, or some learned interaction. Thus a geometry need not be Euclidean geometry. A metric space is one geometry. A weighted graph is another. A kernel matrix defines another. A learned embedding, together with a rule for relating points in the embedding space, induces yet another.

More abstractly, we may write a data geometry as

$$\mathcal{G} = (\mathcal{V}, \mathcal{R}, W),$$

where

- $\mathcal{V}$ is the collection of objects;
- $\mathcal{R} \subseteq \mathcal{V} \times \mathcal{V}$ specifies which relations are available;
- $W : \mathcal{R} \to \mathcal{A}$ describes those relations.

The codomain $\mathcal{A}$ need not be a scalar weight. It may contain a distance or similarity, a direction or displacement, a categorical relation type, a vector of relation attributes, or even an operator specifying how information at one object acts on another.

When relation presence is completely determined by a scalar weight matrix, $\mathcal{R}$ can of course be absorbed into W; we keep it explicit because in many learning problems admissibility and relation strength are different objects. Not every application requires all three components explicitly.

The point is that an object representation by itself does not determine a unique geometry. Representing the objects is not yet the same as choosing their geometry. The same object representation may support many different notions of neighborhood, similarity, interaction, or influence.

1.3.1 The matrix is not the geometry

Consider

$$X = \begin{bmatrix} x_{11} & \cdots & x_{1d} \\ \vdots & & \vdots \\ x_{n1} & \cdots & x_{nd} \end{bmatrix}.$$

The same matrix supports several distinct geometries.

If its rows are regarded as objects,

$$x_i = (x_{i1}, \ldots, x_{id}),$$

then a distance such as

$$d(x_i, x_j)$$

defines a sample geometry.

If the columns are regarded as objects, then quantities such as

$$\operatorname{cov}(X_{\cdot i}, X_{\cdot j})$$

define a feature geometry.

If rows and columns are treated as two different classes of objects, each observed entry x_{ij} becomes a relation between sample i and feature j. The matrix becomes a weighted bipartite graph. This gives a bipartite geometry.

Finally, a map

$$z_i = f_\theta(x_i)$$

may reorganize the samples into a new space. Distances or interactions among the z_i then define a learned latent geometry.

The data matrix therefore does not possess *the* geometry. It admits many geometries.

1.3.2 Completion requires geometry

Suppose that a single entry x_{ij} of a data matrix

$$X \in \mathbb{R}^{n \times d}$$

Table 1.1: Several geometries supported by the same data matrix.

Geometry	Primary objects	Typical relations	Representative methods
Sample geometry	Rows / cases	Distances, neighborhoods, kernels	k-NN, local regression, kernel methods
Feature geometry	Columns / variables	Covariance, correlation, dependence	Linear models, covariance analysis, PCA
Bipartite geometry	Samples and features	Observed entries / weighted edges	Matrix completion, factorization
Learned latent geometry	Embedded states $z = f_\theta(x)$	Learned distances or interactions	Autoencoders, deep networks, transformers

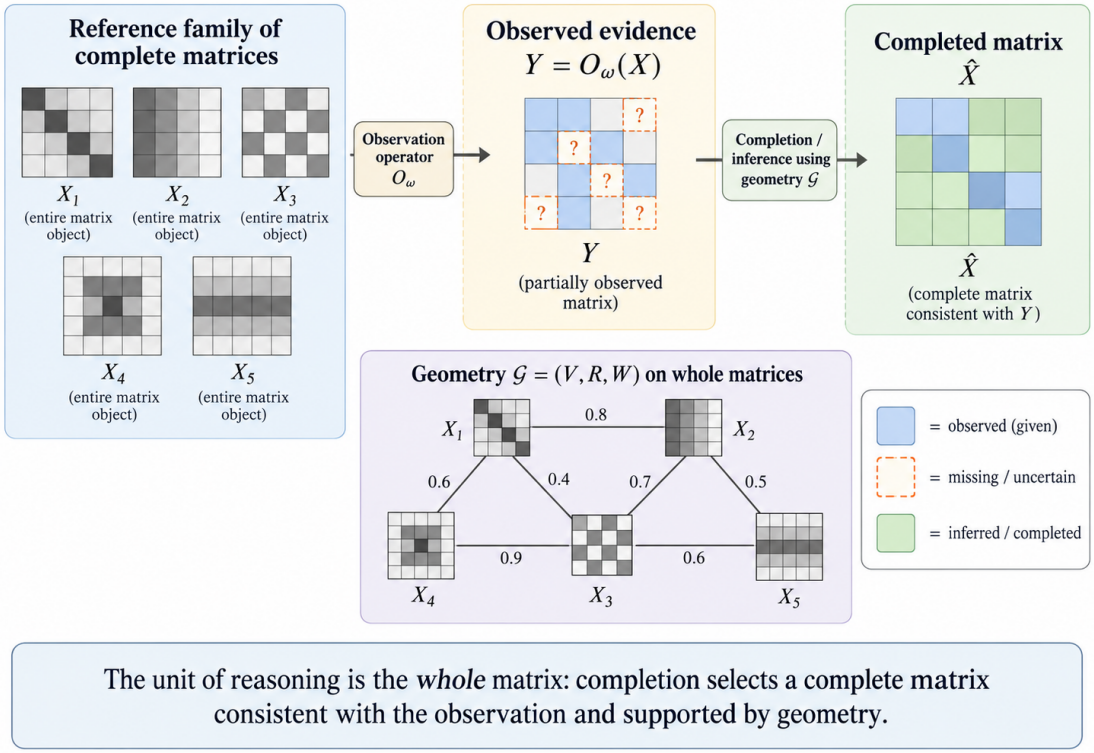


Figure 1.1: Whole-matrix completion in matrix space. The unit of reasoning is a complete matrix object: a partial observation constrains the admissible completions, while a geometry over whole matrices supplies relational support.

is not observed. Writing

$$x_{ij} = ?$$

does not yet tell us how it should be completed. We must also specify *which other objects may provide information about it, and how their relation to the missing entry is to be used.*

This is the operational role of geometry. Recall the three questions:

What are the objects?
Which objects are related?
What does each relation encode or do?

The same missing entry can be completed under quite different answers to these questions.

Completion through sample geometry. First regard the rows

$$x_1, \dots, x_n$$

as the objects. Thus

$$\mathcal{V} = \{x_1, \dots, x_n\}.$$

To estimate x_{ij}, we look for samples whose observed coordinates are similar to those of sample i. Let

$$N_k(i; j)$$

denote the k nearest rows to x_i, with the missing coordinate j excluded when distances are computed.

The relations are therefore sample–sample relations:

$$\mathcal{R} = \{(i, \ell) : \ell \in N_k(i; j)\}.$$

A relation descriptor may be, for example, a distance

$$W(i, \ell) = d(x_i, x_\ell),$$

or a similarity derived from that distance,

$$W(i, \ell) = K(d(x_i, x_\ell)) .$$

A weighted local completion is then

$$\widehat{x}_{ij} = \frac{\displaystyle\sum_{\ell \in N_k(i;j)} W(i, \ell) x_{\ell j}}{\displaystyle\sum_{\ell \in N_k(i;j)} W(i, \ell)}.$$

The unweighted k-nearest-neighbour estimate is obtained by assigning equal weight to all selected neighbours.

The geometry has therefore answered the three questions as follows:

objects	:	samples (rows),
related objects	:	nearby samples,
relation encodes	:	how strongly one sample can provide evidence for another.

Notice that the geometry is not the averaging operation itself. The geometry determines *which rows can supply information and with what influence*; the completion rule specifies how that information is combined. In this case information reaches the missing entry vertically through the matrix:

$$x_{\ell j} \longrightarrow \widehat{x}_{ij}.$$

Completion through feature geometry. Now keep the same missing entry x_{ij}, but regard the columns as the objects:

$$\mathcal{V} = \{x_{\cdot 1}, \ldots, x_{\cdot d}\}.$$

Suppose feature j can be reconstructed from the remaining features by a linear relation

$$x_{\cdot j} \approx \sum_{r \neq j} \beta_{rj} x_{\cdot r}.$$

The relevant relations are now feature–feature relations. For example,

$$\mathcal{R} = \{(r, j) : \beta_{rj} \neq 0\},$$

and we may take

$$W(r, j) = \beta_{rj}.$$

Here W is not a distance. It describes how information in feature r acts on feature j: its magnitude measures the strength of the contribution and its sign its direction. The relation is also generally directed,

$$W(r, j) \neq W(j, r).$$

For row i, the missing value is completed as

$$\widehat{x}_{ij} = \sum_{r \neq j} W(r, j) x_{ir}.$$

The three questions now receive different answers:

$$\begin{aligned} \text{objects} &: \text{features (columns)}, \\ \text{related objects} &: \text{features informative about one another}, \\ \text{relation encodes} &: \text{how one feature contributes to another}. \end{aligned}$$

Information now reaches the same missing entry horizontally through the matrix:

$$x_{ir} \longrightarrow \widehat{x}_{ij}.$$

Same hole, different geometry. The two procedures attempt to fill exactly the same missing value, but they obtain the required information from different relational structures:

sample geometry	$x_{\ell j} \longrightarrow x_{ij}$	information across rows,
feature geometry	$x_{ir} \longrightarrow x_{ij}$	information across columns.

Thus, for a completion problem $x_{i,j} =?$ geometry supplies the additional structure needed to solve it: it identifies the objects from which information may come and describes the relations through which that information can act.

This distinction will recur throughout the book. Different learning methods often address the same completion problem while making very different choices about the objects, the admissible relations among them, and the information carried by those relations.

1.4 A geometry may be assumed, constructed, or learned

Once the objects and their possible relations have been made explicit, a natural question is:

Where does the geometry used by a learning method come from?

There is no single answer. In some methods the geometry is chosen before the data are examined; in others it is computed from the data; in still others it is modified by learning so as to make the task easier.

These cases are not mutually exclusive categories. A method may combine all three. The distinction is useful because it tells us which part of the relational structure is supplied by the designer, which part is extracted from the observations, and which part is adapted to the learning objective.

Recall that we describe a geometry schematically as

$$\mathcal{G} = (\mathcal{V}, \mathcal{R}, W),$$

where $\mathcal{V}$ contains the objects, $\mathcal{R}$ specifies which objects may be related, and

$$W : \mathcal{R} \longrightarrow \mathcal{A}$$

describes the relation. Depending on the problem, W may encode a distance, a similarity, a direction, a collection of relation features, or a transformation through which one object influences another.

1.4.1 Assumed geometry

The simplest case is one in which the relevant relational structure is chosen in advance.

Consider k-nearest-neighbour prediction. The objects are the samples,

$$\mathcal{V} = \{x_1, \ldots, x_n\}.$$

Before any prediction can be made, we must decide what it means for two samples to be close. For example, we may choose Euclidean distance,

$$W(i, j) = d(x_i, x_j) = \|x_i - x_j\|_2.$$

The admissible relation set may contain all sample pairs,

$$\mathcal{R} = \{(i, j) : i \neq j\},$$

while the completion rule uses only those relations for which j belongs to the selected neighbourhood $N_k(i)$.

Thus the three components are

objects	:	samples,
relations	:	sample–sample comparisons,
relation descriptor	:	distance or similarity.

The important point is that the data do not by themselves tell us to use Euclidean distance. We have supplied that choice.

A different assumed metric,

$$d_A(x_i, x_j)^2 = (x_i - x_j)^\top A(x_i - x_j),$$

or cosine similarity,

$$s(x_i, x_j) = \frac{x_i^\top x_j}{\|x_i\| \, \|x_j\|},$$

would give a different geometry on exactly the same object representation. Different neighbours could then become relevant, and the resulting completion could change.

This illustrates an important distinction:

Principle 1.4 (Representation is not geometry) *An object representation specifies how objects are encoded. It does not, by itself, specify how those represented objects are related.*

For k-nearest neighbours, therefore, the geometry is largely *assumed*: the designer chooses the representation, the metric, and usually the neighbourhood rule before the completion is performed.

1.4.2 Geometry constructed from data

A geometry may instead be obtained by applying a fixed construction to the observed data.

Principal component analysis provides a useful example. Let the columns of a centered matrix (each column has mean 0)

$$X \in \mathbb{R}^{n \times d}$$

be regarded as the objects,

$$\mathcal{V} = \{x_{\cdot 1}, \ldots, x_{\cdot d}\}.$$

Every pair of features may be related,

$$\mathcal{R} = \{(r, s) : 1 \le r, s \le d\},$$

and their relation can be described by covariance,

$$W(r, s) = \Sigma_{rs}, \qquad \Sigma = \frac{1}{n} X^\top X.$$

Here $W(r, s)$ tells us how variations in feature r and feature s co-vary across the observed samples. The geometry is therefore not supplied as a fixed numerical matrix before seeing the data: it is *constructed* from them.

PCA then extracts directions

$$v_1, \ldots, v_d$$

from the eigendecomposition

$$\Sigma v_j = \lambda_j v_j.$$

Retaining the leading directions produces a new representation

$$z = V_k^\top x$$

in which relations among observations may be simpler or more concentrated.

There are therefore two related constructions here. First, the data induce a feature geometry through covariance. Second, that geometry is used to construct a new representation.

Schematically,

$$X \longrightarrow \Sigma \longrightarrow V_k \longrightarrow z.$$

The procedure for constructing the geometry is fixed in advance, but the geometry itself depends on the observed data.

This distinction is useful when comparing PCA with the previous example:

k-NN	:	choose a relation rule, then use it on the data,
PCA	:	use a fixed rule to construct relations from the data.

In the broad statistical sense PCA is of course a learning method. Here "constructed" is not meant to deny that. It emphasizes that the form of the construction—covariance followed by eigendecomposition—is fixed, rather than adapted to a particular prediction target.

1.4.3 Learned geometry

A third possibility is that the relations themselves depend on parameters that are adjusted by a learning objective.

A particularly transparent example is metric learning. Suppose again that the objects are samples,

$$\mathcal{V} = \{x_1, \dots, x_n\},$$

but now let their distance be

$$W_\theta(i,j) = d_\theta(x_i, x_j).$$

For example,

$$d_\theta(x_i, x_j)^2 = (x_i - x_j)^\top A_\theta (x_i - x_j),$$

where the positive semidefinite matrix A_θ is learned from data. Training may encourage samples that should support one another to become close and samples that should be distinguished to become distant.

Now the geometry itself changes during learning:

$$\mathcal{G}_{\theta_0} \longrightarrow \mathcal{G}_{\theta_1} \longrightarrow \cdots \longrightarrow \mathcal{G}_{\theta_*}.$$

The objects need not have changed. What has changed is the relation by which they are compared.

Deep representation learning generalizes this idea. Let

$$z = f_\theta(x)$$

be a learned representation. A simple geometry in representation space might then be

$$W_\theta(i,j) = \|f_\theta(x_i) - f_\theta(x_j)\|_2 .$$

Although Euclidean distance is fixed in the z-coordinates, the effective relation between the original objects changes whenever f_θ changes. Thus learning the representation also changes the geometry induced on the original samples.

Two inputs that were far apart in the original representation may become nearby,

$$\|x_i - x_j\| \text{ large}, \qquad \|f_\theta(x_i) - f_\theta(x_j)\| \text{ small},$$

because the learned representation has discovered that they should provide similar evidence for the task.

Conversely, two inputs that appear close in raw coordinates may be separated if their task-relevant roles differ.

This is the sense in which representation learning can be understood geometrically:

Principle 1.5 (Representation learning) *Representation learning changes the representation of the objects and thereby changes the geometry through which they can constrain one another.*

It is therefore slightly imprecise to say simply that a neural network "learns a geometry." More precisely, it learns a representation

$$\rho_\theta : x \mapsto f_\theta(x),$$

and this representation, together with a relation rule in representation space, induces a geometry on the original objects.

1.4.4 Why the distinction matters for completion

The three cases differ in the origin of the relations, but they serve the same operational purpose. When some component is open, geometry determines which available objects may provide information about it and how that information can act.

We may summarize the progression as

assumed geometry	:	relations supplied before learning,
constructed geometry	:	relations computed from observed data,
learned geometry	:	relations modified according to a learning objective.

For completion, the practical question is always the same:

From which objects may information arrive, and what relation determines its influence?

The answer changes as we move through the book. Local methods begin with largely assumed geometries. Linear and latent methods construct useful relations from data. Methods for representation learning increasingly adapt the representation—and hence the induced geometry—to the task itself.

The progression is therefore not simply from "simple" to "complex" models. It is also a progression in how much of the geometry is supplied by us and how much is discovered by the machine.

1.5 Supervised learning as completion

The completion formulation is particularly transparent for supervised learning. Suppose we observe samples

$$(x_i, y_i), \qquad i = 1, \ldots, n.$$

Formally, we may regard the target as an additional component of the represented data object:

$$\widetilde{X} = [X \mid y].$$

This augments the object that may be partially observed; it does not by itself define a new geometry. For training samples, both x_i and y_i are observed. For a new sample,

$$(x_*, ?),$$

the target component is masked. Prediction is therefore

$$(x_*, ?) \quad \longrightarrow \quad (x_*, \widehat{y}_*).$$

Classification and regression differ in the nature of y and in the completion rule, but the structural operation is the same. In classification,

$$y \in \{1, \ldots, K\},$$

whereas in regression,

$$y \in \mathbb{R}^q.$$

Both can be understood as target completion. This does not abolish the distinction between classification and regression. Their output spaces, losses, probabilistic interpretations, and evaluation criteria remain different. What the completion viewpoint provides is a common structural description beneath those differences.

1.6 Changing the observation changes the problem

An important consequence follows immediately. A learning problem is determined not only by the underlying object but also by *how that object is observed*. If we retain the same object X and change the observation operator $\mathcal{O}$, we can obtain quite different tasks.

When the observation operator is a mask,

$$Y = \mathcal{O}_M(X) = M \odot X,$$

changing M changes which components are available. Mask a label,

$$(x, ?),$$

and the task is prediction. Mask arbitrary feature values,

$$(x_1, \ldots, ?, \ldots, x_d),$$

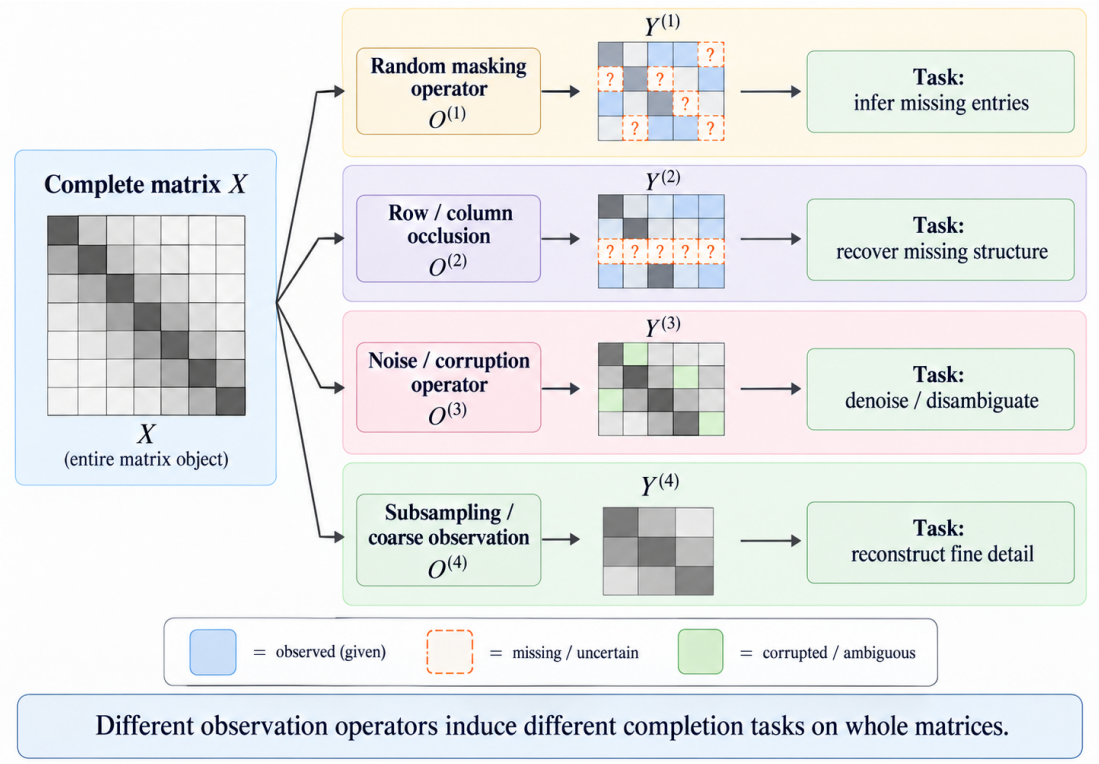


Figure 1.2: Changing the observation operator changes the matrix-completion problem. Random masking, structured row or column occlusion, corruption, and coarse observation leave different information unresolved and therefore induce different completion tasks.

and the task becomes imputation. Mask a contiguous spatial region in an image and the task becomes inpainting. Reveal only a prefix of a sequence,

$$x_1, \dots, x_t, ?, \dots, ?,$$

and the unresolved part is a continuation or forecast.

But not every change of observation is a change of mask. If

$$Y = X + \varepsilon,$$

all coordinates may be present and yet corrupted; the task is denoising. If

$$Y = KX + \varepsilon,$$

the observation is blurred or mixed. If

$$Y = AX,$$

the object has been projected or compressed.

Thus the more general principle is

$$\boxed{\text{same underlying object} + \text{different observation process} \implies \text{different inference problem.}}$$

Principle 1.6 (Observation is part of the problem) *The observation operator is part of the definition of a learning problem. A mask is one important special case.*

A learning experiment is therefore incompletely specified unless we state not only the data and the model, but also how the information presented to the learner was obtained.

1.7 Learning the completion rule and using it

The completion viewpoint introduces an important distinction that should not be blurred. A fitted model can be used to complete an unresolved quantity, but machine learning must first construct or adapt the rule that performs that completion.

Let

$$Y = \mathcal{O}(X)$$

be the available observation and let $T(X)$ denote the quantity we want to infer. Once a completion rule $\widehat{\mathcal{C}}$ has been obtained, use of the model has the form

$$Y \xrightarrow{\widehat{\mathcal{C}}} \widehat{T(X)}.$$

Learning is the different operation by which finite data determine the rule:

$$D_{\text{train}} \xrightarrow{\text{learning}} \widehat{\mathcal{C}}.$$

For supervised learning this becomes

$$\{(x_i, y_i)\}_{i=1}^n \longrightarrow \widehat{\mathcal{C}}, \qquad (x_*, ?) \xrightarrow{\widehat{\mathcal{C}}} \widehat{y}_*.$$

The first arrow constructs the completion mechanism; the second uses it. Calling both arrows "completion" would hide a real computational distinction rather than clarify it.

This also explains why a fully observed training example can matter even though there is nothing externally missing from it. The pair

$$(x_i, y_i)$$

supplies evidence from which the completion rule can be changed. In an online setting the same idea can be seen particularly clearly:

$$(x_i, ?) \longrightarrow \widehat{y}_i \longrightarrow (x_i, y_i).$$

The first presentation asks the current rule for a completion; revealing y_i then provides evidence for modifying that rule.

Principle 1.7 (Learning and use) *Learning constructs or modifies a completion rule from experience. Use applies the current rule to a partially observed object. Some learning systems interleave these operations, but they should not be confused.*

This distinction will remain deliberately simple in the early chapters. Later, when we study online learning, sequence models, and adaptive machines, we will return to the question of how tightly learning and use can be coupled.

1.8 Train, test, and generalization

The train/test split makes the distinction operational.

Training data are used to construct a rule,

$$D_{\text{train}} \longrightarrow \widehat{\mathcal{C}},$$

while test data ask whether that rule supports new completions,

$$Y_* \xrightarrow{\widehat{\mathcal{C}}} \widehat{T(X_*)}.$$

Generalization is therefore not merely the ability to fill a hole. It is the ability to complete a new case using structure acquired from earlier finite experience:

generalization = successful completion on information not used to fit the rule.

Two limitations of information meet here and should not be confused. The observation operator $\mathcal{O}_*$ determines what is unavailable or unreliable in the current test object. The finite training sample limits the evidence from which the completion rule itself was learned. The first is an object-level observation constraint; the second is a learning-level limitation.

The train/test split is therefore an evaluation protocol: it separates the information allowed to shape the rule from the information used to judge whether the rule transfers. The statistical questions of expected risk, sampling, capacity, bias, variance, and overfitting remain essential; the completion viewpoint simply gives us a common description of what the fitted rule is being asked to do.

1.9 From completion to regulated completion

A final issue already appears at the beginning of the book. A conventional predictor is expected to produce an output whenever an input is supplied. But not every partial observation licenses a reliable completion.

The available evidence may be

- insufficient,
- contradictory,
- outside the training distribution,
- compatible with several distinct completions,
- or expressed in a geometry in which the model has little support.

A mature learning system must therefore do more than complete. It must assess the conditions under which completion is justified.

At the most general level we can write

$$\boxed{\text{observe} \longrightarrow \text{complete} \longrightarrow \text{assess.}}$$

The observation process determines what information reaches the learner; the completion rule uses the current geometry to infer what remains unresolved; assessment asks whether the result is sufficiently supported to be used.

For masked dynamical models studied later, this will acquire the more specific form

$$\boxed{\text{clamp} \longrightarrow \text{relax} \longrightarrow \text{complete} \longrightarrow \text{regulate.}}$$

There, clamping fixes the observed coordinates, relaxation propagates information through the current geometry or dynamics, and regulation decides whether the resulting completion should be committed to.

The early chapters will mostly concentrate on observation, geometry, and completion. Regulation will become increasingly important when we reach uncertainty, distribution shift, iterative models, and adaptive systems.

1.10 A common description of a learning problem

We can now state the viewpoint precisely. Machine learning will be studied as the construction and persistent modification of structures that support completion of partially observed objects.

A learning problem can be described by four elements:

$$\boxed{(\mathcal{X}, \mathcal{O}, \mathcal{G}, \mathcal{C})}$$

where

- $\mathcal{X}$ specifies the objects under consideration;
- $\mathcal{O}$ specifies how those objects are observed;
- $\mathcal{G}$ specifies the relations through which available information can influence unresolved quantities;

- $\mathcal{C}$ specifies how that information is combined to produce a completion.

The task usually fixes much of $\mathcal{X}$ and $\mathcal{O}$. The modeller and the learning procedure then choose, construct, or learn the geometry and the completion rule.

This gives us a common language in which to compare algorithms that are usually taught separately. Nearest-neighbour methods complete through local sample geometry. Linear models complete through fitted directions in feature space. Kernel methods replace the original geometry with an induced similarity geometry. Principal-component and low-rank methods construct latent completion spaces. Neural networks progressively transform the representation and hence its geometry. Attention dynamically constructs relations among elements of the current object. Probabilistic models represent not only candidate completions but uncertainty over them. Generative models learn to complete severely degraded or partially specified states.

The algorithms remain different. Their inductive biases remain different. Their statistical properties remain different. But they can now be asked the same set of questions:

1. What is the underlying object?
2. How is it observed, corrupted, masked, or compressed?
3. What quantity remains unresolved?
4. What are the relevant objects for the geometry?
5. Which of those objects are related, and what does each relation encode?
6. How are those relations used to produce the completion?
7. When should the resulting completion be trusted?

Those questions will provide the conceptual spine of the chapters that follow.

1.11 One matrix, one hidden entry, four geometries

It is useful to make the preceding discussion numerical before introducing any particular learning algorithm. Consider

$$X = \begin{bmatrix} 1.00 & 1.010 & 0.10 \\ 1.20 & 1.190 & -0.10 \\ 4.80 & 4.815 & 0.15 \\ 5.00 & 4.985 & -0.15 \end{bmatrix}.$$

Now hide a single entry,

$$x_{21} = 1.20,$$

and pretend that its value is unknown:

$$\widetilde{X} = \begin{bmatrix} 1.00 & 1.010 & 0.10 \\ ? & 1.190 & -0.10 \\ 4.80 & 4.815 & 0.15 \\ 5.00 & 4.985 & -0.15 \end{bmatrix}.$$

Every construction below is asked to solve exactly the same problem:

$$\boxed{\text{estimate the missing entry } x_{21}.}$$

What changes is the geometry: which objects we regard as related, what the relation encodes, and how that relation is converted into a numerical completion.

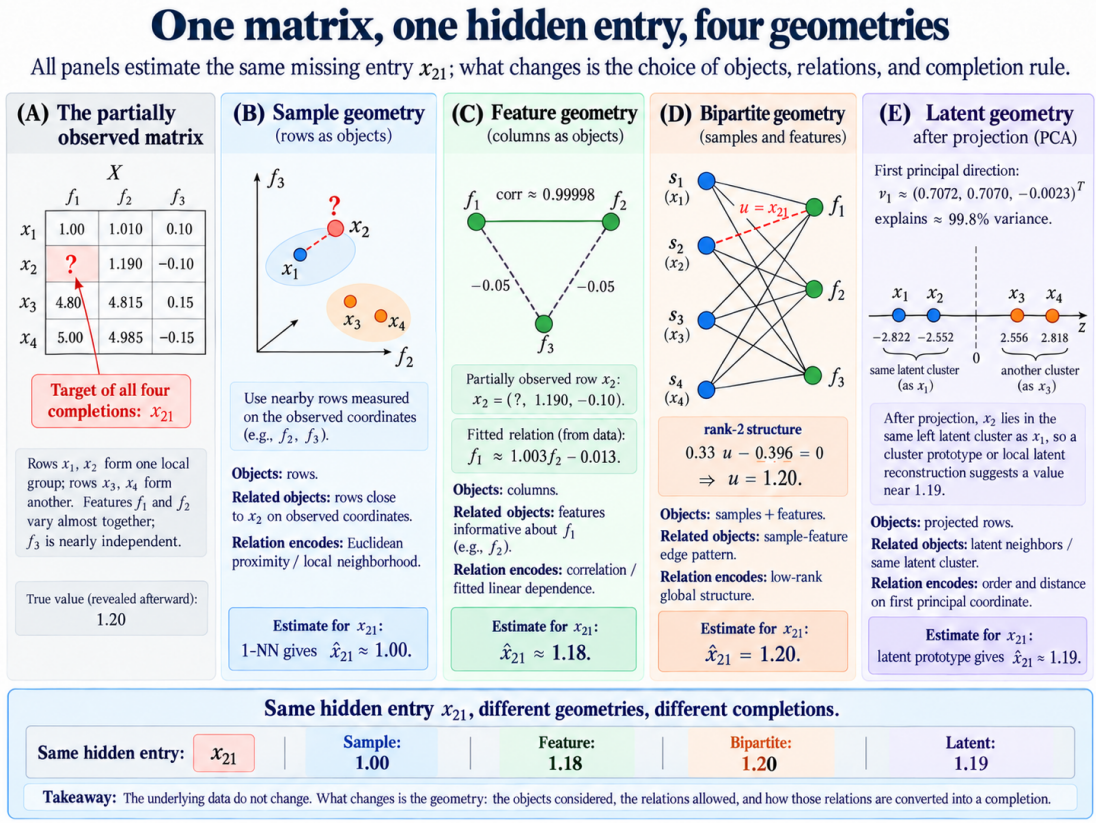


Figure 1.3: The same hidden entry x_{21} completed under four different geometries. The data and the missing quantity are fixed; only the objects, relations, and completion mechanism change.

1.11.1 Rows as objects: sample geometry

First take the rows as the objects:

$$\mathcal{V} = \{x_1, x_2, x_3, x_4\}.$$

Because x_{21} is hidden, row x_2 must be compared with the other rows using only the observed coordinates f_2 and f_3. The corresponding distances are

$$\begin{aligned} d(x_2, x_1) &= \sqrt{(1.190 - 1.010)^2 + (-0.10 - 0.10)^2} \approx 0.269, \\ d(x_2, x_3) &\approx 3.634, \\ d(x_2, x_4) &\approx 3.795. \end{aligned}$$

Thus x_1 is by far the nearest row. A one-nearest-neighbour completion copies the missing first coordinate from that row:

$$\boxed{\widehat{x}_{21}^{\text{sample}} = x_{11} = 1.00.}$$

The geometry is

$$\begin{aligned} \text{objects} &: \text{rows}, \\ \text{related objects} &: \text{rows close on the observed coordinates}, \\ \text{relation encodes} &: \text{Euclidean proximity}. \end{aligned}$$

Information reaches the missing entry from the same feature of a nearby sample:

$$x_{11} \longrightarrow \widehat{x}_{21}.$$

1.11.2 Columns as objects: feature geometry

Now regard the columns as the objects:

$$\mathcal{V} = \{x_{\cdot 1}, x_{\cdot 2}, x_{\cdot 3}\}.$$

Using the fully observed rows $1, 3, 4$, the first two features are almost linearly related. Fitting f_1 from f_2 on those rows gives

$$x_{\cdot 1} \approx 1.003\, x_{\cdot 2} - 0.013.$$

The observed value $x_{22} = 1.190$ therefore gives

$$\widehat{x}_{21}^{\text{feature}} = 1.003(1.190) - 0.013 \approx 1.180.$$

Hence

$$\boxed{\widehat{x}_{21}^{\text{feature}} \approx 1.18.}$$

Here the geometry is

$$\begin{aligned} \text{objects} &: \text{features}, \\ \text{related objects} &: \text{features informative about one another}, \\ \text{relation encodes} &: \text{fitted linear dependence}. \end{aligned}$$

Information now travels across columns within the same sample:

$$x_{22} \longrightarrow \widehat{x}_{21}.$$

Notice the difference from the previous completion. The missing quantity is identical, but the sample geometry produced 1.00, whereas the feature geometry produces approximately 1.18.

1.11.3 Samples and features as objects: bipartite geometry

A third view treats samples and features as two classes of nodes,

$$\mathcal{S} = \{s_1, \dots, s_4\}, \qquad \mathcal{F} = \{f_1, f_2, f_3\},$$

with each observed entry x_{ij} represented as the weight of the edge between s_i and f_j.

In this view the missing value x_{21} is a missing edge weight. Suppose that the full sample–feature matrix is generated by two latent factors, so that its rank is 2. Then every 3×3 minor must have zero determinant. Writing $u = x_{21}$ and using the first three rows,

$$\det \begin{bmatrix} 1.00 & 1.010 & 0.10 \\ u & 1.190 & -0.10 \\ 4.80 & 4.815 & 0.15 \end{bmatrix} = 0.33u - 0.396.$$

The rank–2 constraint gives

$$0.33u - 0.396 = 0,$$

hence

$$\boxed{\widehat{x}_{21}^{\,\text{bipartite}} = 1.20.}$$

The geometry is now

objects	:	samples and features,
related objects	:	sample–feature pairs,
relation encodes	:	observed edge values constrained by a low-rank global structure.

Unlike the first two methods, the completion is not obtained from one nearby row or one directly related feature. It is forced by a global constraint on the entire sample–feature relation structure.

1.11.4 Projected rows: latent geometry

Finally, construct a one-dimensional latent representation from the complete rows $1, 3, 4$. Center those rows and fit a one-component PCA model. The fitted mean and first principal direction are approximately

$$\mu = \begin{bmatrix} 3.600 \\ 3.603 \\ 0.033 \end{bmatrix}, \qquad v_1 = \begin{bmatrix} 0.7080 \\ 0.7060 \\ -0.0201 \end{bmatrix}.$$

The first component explains about 99.8% of the variance of the three complete rows.

For the incomplete row x_2, we cannot compute its latent coordinate from the missing first feature. Instead we choose the latent coordinate z_2 that best matches the two observed coordinates f_2 and f_3:

$$z_2 = \arg\min_z \left\| \begin{bmatrix} 1.190 \\ -0.10 \end{bmatrix} - \left(\begin{bmatrix} 3.603 \\ 0.033 \end{bmatrix} + z \begin{bmatrix} 0.7060 \\ -0.0201 \end{bmatrix} \right) \right\|^2.$$

This gives

$$z_2 \approx -3.410.$$

Reconstructing the missing first coordinate from the latent line,

$$\widehat{x}_{21}^{\text{latent}} = 3.600 + 0.7080\, z_2 \approx 1.186,$$

so

$$\boxed{\widehat{x}_{21}^{\text{latent}} \approx 1.19.}$$

Here the geometry is

objects	:	rows represented by latent coordinates,
related objects	:	points close along the learned principal direction,
relation encodes	:	proximity in the projected representation.

This completion is close to the feature-based estimate, but it is obtained differently: the feature model predicts f_1 directly from f_2, whereas the PCA model first constrains the whole row to lie near a learned one-dimensional latent structure and reconstructs the missing coordinate from that structure.

1.11.5 Same hole, different geometries

The four estimates are now directly comparable:

$$\boxed{\widehat{x}_{21}^{\text{sample}} = 1.00, \qquad \widehat{x}_{21}^{\text{feature}} \approx 1.18, \qquad \widehat{x}_{21}^{\text{bipartite}} = 1.20, \qquad \widehat{x}_{21}^{\text{latent}} \approx 1.19.}$$

The true value, revealed only after the four completions have been produced, is

$$x_{21} = 1.20.$$

The important point is not that one geometry happens to win on this deliberately constructed example. It is that the *same observed matrix with the same missing entry* leads to different answers because each geometry permits information to reach the hole through a different relational structure.

Table 1.2: Four completions of the same hidden entry x_{21}.

Geometry	Objects	Relation used	Estimate
Sample	rows	nearest row on observed coordinates	1.00
Feature	columns	fitted dependence $f_2 \to f_1$	1.18
Bipartite	samples and features	global rank–2 edge structure	1.20
Latent	projected rows	one-dimensional PCA structure	1.19

This is the operational meaning of geometry used throughout the book:

$$\boxed{\text{same unresolved quantity} \;+\; \text{different relational structure} \;\Longrightarrow\; \text{different completion.}}$$

1.12 Where we begin

We will begin with the simplest possible completion mechanisms.

A global mean ignores almost all geometry. Nearest-neighbour methods introduce a fixed local geometry. Linear methods fit global directions in feature space. Low-rank methods infer latent structure. Neural methods learn increasingly flexible representations. Probabilistic methods make uncertainty part of the completion itself. Later, regulated completion will ask not only what should be produced but whether the current geometry provides enough support to produce it at all.

The progression is therefore not simply

$$k\text{-NN} \longrightarrow \text{linear models} \longrightarrow \text{PCA} \longrightarrow \text{neural networks}.$$

It is

$$\boxed{\begin{array}{c}\text{fixed completion} \longrightarrow \text{geometric completion} \\ \longrightarrow \text{learned geometry} \longrightarrow \text{regulated completion}\end{array}}.$$

That progression is the subject of this book.

1.13 Questions for Study

1. Give an example of a machine-learning task for which the "unresolved information" viewpoint is illuminating, and one for which it is mostly cosmetic.
2. Explain the difference between learning a completion rule and using that rule. Why is a fully observed training example still informative even though nothing is externally missing?
3. For a data matrix $X \in \mathbb{R}^{n\times d}$, define two different sample geometries and explain how they can produce different completions from the same observation.
4. Show how classification can be written as completion of an appended target component. Why does appending the target not, by itself, define a new geometry?
5. Give one masked, one noisy, and one projected observation operator on the same underlying object. What different inference problems do they create?
6. In the numerical example of Section 1.11, explain why the feature-regression completion and the rank–2 completion of x_{21} are based on different geometrical assumptions.
7. Explain why learning a representation can change the induced geometry even when Euclidean distance is still used after the transformation.
8. State a criterion by which the completion viewpoint itself could be judged unhelpful for a given method.

1.14 References and Further Reading

The following works provide broad foundations for the statistical and probabilistic ideas of the chapter, and for the later move toward learned representations.

- Hastie et al. (2009) — a unified statistical-learning treatment of supervised and unsupervised methods, especially useful for comparing inductive biases rather than only algorithms.
- Bishop (2006) — a probabilistic treatment of pattern recognition that makes latent variables, uncertainty, and conditional prediction explicit.
- Murphy (2022) — a modern probabilistic-machine-learning reference with broad coverage of the models used throughout this book.
- Goodfellow et al. (2016) — a standard account of deep learning, representation learning, optimization, and generative modelling.
- Bengio et al. (2013) — a useful bridge from hand-designed features to learned representations and the idea that learning changes the geometry in which downstream tasks are solved.

Chapter 2

Local Completion: Neighbours and Kernels

2.1 The first geometric answer

The previous chapter described a learning problem through four ingredients,

$$(\mathcal{X}, \mathcal{O}, \mathcal{G}, \mathcal{C}),$$

and separated two questions that are easy to conflate:

1. how the underlying object is observed, through $\mathcal{O}$;
2. how the available information is related, through $\mathcal{G}$.

We now study the simplest nontrivial geometric answer.

To estimate an unresolved value, use observed objects that are nearby.

This is the idea of local completion. In this chapter the objects will usually be *samples*, and the observation pattern will usually be simple masking: a target or feature value is hidden while other coordinates remain visible. The broader observation operators of section 1.2—noise, blur, projection, and so on—will return later. Starting with masks lets us see the local geometry without mixing it with a difficult observation model.

Let

$$\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n, \qquad x_i \in \mathbb{R}^d,$$

be observed training examples. For a new query x_*, the target component is not observed:

$$(x_*, ?).$$

As in section 1.7, two operations must be kept distinct. The training data are used to choose or fit whatever the local rule needs—for example preprocessing, a metric or metric parameters, and a neighbourhood scale. Once these choices are fixed, prediction uses the resulting rule to complete the current query.

For a fixed query, the local geometry can be made explicit in the notation of Chapter 1. Take

$$\mathcal{V}_* = \{x_*, x_1, \dots, x_n\}$$

as the sample objects. The query may be compared with the stored samples through admissible relations

$$\mathcal{R}_* = \{(x_*, x_i) : i = 1, \dots, n\},$$

and a relation descriptor such as

$$W(x_*, x_i) = d(x_*, x_i)$$

records their distance. A neighbourhood rule selects which of those relations will actually supply evidence, and an aggregation rule converts the corresponding observed targets into a completion.

Thus a local method has the schematic form

$$x_* \xrightarrow[\text{relation descriptor}]{\text{sample geometry}} N(x_*) \xrightarrow{\text{aggregation}} \widehat{y}_*.$$

This decomposition is important. The metric or similarity does not itself predict the target. It determines which sample relations are relevant. The completion rule then says how information carried by those related samples is combined.

Principle 2.1 (Local completion) *A local completion method estimates an unresolved quantity from observed sample objects selected or weighted according to their geometric relation to the current query.*

Nearest-neighbour methods are therefore useful here for more than their historical role as simple classifiers and regressors. They provide the first case in which the objects, relations, relation descriptors, and completion readout can all be seen directly.

2.2 Before locality: global completion baselines

The meaning of locality is clearest if we begin with completion rules that do not distinguish one sample from another.

Suppose $y_i \in \mathbb{R}$. The most elementary regression completion is the global mean

$$\widehat{y}_* = \bar{y} = \frac{1}{n} \sum_{i=1}^{n} y_i.$$

For categorical targets, the analogous completion is the global mode

$$\widehat{y}_* = \operatorname{mode}(y_1, \dots, y_n).$$

These rules use information learned from the training sample, but at use time they ignore the position of x_*. Every query receives the same completion. In geometric terms, every stored sample has the same influence on every query.

A common expression for regression makes the transition to locality explicit:

$$\widehat{y}_* = \frac{\sum_{i=1}^{n} w_i(x_*) y_i}{\sum_{i=1}^{n} w_i(x_*)}.$$

The global mean corresponds to

$$w_i(x_*) = 1 \qquad \text{for every } i, x_*.$$

Local methods instead derive $w_i(x_*)$, or a hard neighbourhood, from the relation between x_i and x_*.

It is useful to separate four choices:

1. the representation in which the sample objects are described;
2. the relation descriptor used to compare them, such as a distance or similarity;
3. the rule that selects or weights related samples;
4. the readout that combines their observed targets.

The first three determine what evidence is locally available. The fourth converts that evidence into the quantity required by the task. This is why the same local geometry can later support classification, regression, or feature imputation.

2.3 Metrics create neighbourhoods

To speak of a nearby sample we need a rule for comparing two sample objects.

Definition 2.2 (Metric) *A metric on a set $\mathcal{X}$ is a function*

$$d : \mathcal{X} \times \mathcal{X} \to [0, \infty)$$

satisfying, for all $x, y, z \in \mathcal{X}$,

$$\begin{aligned} &d(x, y) = 0 \iff x = y, \\ &d(x, y) = d(y, x), \\ &d(x, z) \le d(x, y) + d(y, z). \end{aligned}$$

A metric is not merely a technical accessory. Once chosen, it defines which points count as similar for local completion.

For numerical vectors, the most common choice is Euclidean distance,

$$d_2(x, z) = \|x - z\|_2 = \left(\sum_{j=1}^{d} (x_j - z_j)^2 \right)^{1/2}.$$

Other common distances include Manhattan distance,

$$d_1(x, z) = \sum_{j=1}^{d} |x_j - z_j|,$$

and the general Minkowski family,

$$d_p(x, z) = \left(\sum_{j=1}^{d} |x_j - z_j|^p \right)^{1/p}.$$

A positive definite quadratic form gives the Mahalanobis-type metric

$$d_A(x, z) = \sqrt{(x - z)^\top A(x - z)}, \qquad A \succ 0.$$

When $A = I$, this reduces to Euclidean distance. If $A \succeq 0$ is singular, the same expression is a pseudometric: distinct vectors can have zero distance because some directions have been collapsed. A nontrivial A changes the relative importance of directions in feature space. Thus even before fitting a predictor, a choice of relation descriptor may encode a substantial inductive bias.

In the notation of Chapter 1, the sample objects are the points x_i, admissible relations are sample–sample or query–sample comparisons, and $W(i, j) = d(x_i, x_j)$ is one possible relation descriptor. The metric is therefore an ingredient of the geometry, not the geometry by itself.

2.3.1 Neighbourhoods

Given a metric d, an ε-ball around x is

$$B_\varepsilon(x) = \{z \in \mathcal{X} : d(x, z) \le \varepsilon\}.$$

This induces a radius-based neighbourhood: all observed samples inside the ball are locally available to the query.

The alternative is a rank-based neighborhood. Let

$$d_{(1)}(x_*) \le d_{(2)}(x_*) \le \cdots \le d_{(n)}(x_*)$$

be the ordered training distances from x_*. The k-nearest-neighbour set is

$$N_k(x_*) = \{i : d(x_i, x_*) \le d_{(k)}(x_*)\},$$

with an explicit tie rule when necessary.

The two constructions adapt in opposite ways:

- fixed ε keeps the geometric radius constant but allows the number of neighbors to vary;
- fixed k keeps the amount of evidence approximately constant but allows the radius to vary.

This distinction already exposes a density issue. In a dense region, k-NN uses a small radius. In a sparse region, it expands until k samples have been found. The method therefore adapts its spatial scale to local sample density.

Definition 2.3 (Neighbourhood scale) *The neighbourhood scale of a query is the spatial extent from which a local method draws evidence. In k-NN it can be measured by the distance $d_{(k)}(x_*)$ to the k-th neighbor; in radius methods it is prescribed directly by ε.*

The scale is not a minor tuning detail. It controls how much local structure is preserved and how much evidence is pooled.

2.4 The geometry is not given by the numbers alone

A numerical matrix does not determine a meaningful distance automatically. Euclidean distance treats all coordinates as commensurable. This assumption is often false.

Suppose a data matrix has two features:

$$x = (\text{age in years}, \text{income in euros}).$$

If income ranges over tens of thousands while age ranges over tens, raw Euclidean distance will be dominated by income. The algorithm will behave as though age were almost irrelevant even if age is strongly predictive.

A common remedy is standardization,

$$x'_{ij} = \frac{x_{ij} - \mu_j}{s_j},$$

where μ_j and s_j are estimated from the training data. Then Euclidean distance is computed in the standardized coordinates.

This simple operation deserves a precise interpretation. Standardization first changes the *representation*,

$$\rho(x) = D^{-1}(x - \mu), \qquad D = \operatorname{diag}(s_1, \ldots, s_d),$$

and Euclidean distance in the transformed representation induces

$$\|\rho(x) - \rho(z)\|_2^2 = (x - z)^\top D^{-2}(x - z)$$

on the original objects. Thus the representation and the relation rule remain distinct, exactly as in Chapter 1: changing the representation changes the geometry induced by the same Euclidean rule.

The means and scales must be estimated from the training data only. Recomputing them using test samples would let test information participate in the fitted completion rule and would invalidate the train/test separation.

Principle 2.4 (Preprocessing can change induced geometry) *A transformation of the object representation changes the induced geometry whenever it changes the distances, similarities, neighbourhoods, or directions used by the completion rule.*

The same is true of feature selection, monotone transformations, categorical encodings, dimensionality reduction, and whitening. Local methods make this particularly visible because their predictions can change abruptly when neighborhoods change.

2.4.1 Mixed and structured data

Real datasets may contain categorical variables, binary indicators, ordered categories, text, graphs, or other non-Euclidean objects. In those cases a sensible local geometry must respect the object type.

For binary vectors, Hamming distance may be useful:

$$d_H(x, z) = \sum_{j=1}^{d} \mathbf{1}[x_j \neq z_j].$$

For strings one may use edit distances. For probability distributions, a divergence, a transport distance, or another task-appropriate dissimilarity may be useful. For graphs, relations can be defined through paths, graph kernels, or an embedding together with a relation rule.

Not all useful dissimilarities satisfy the metric axioms. Nearest-neighbour ideas extend beyond strict metrics as long as the chosen relation provides a meaningful ranking or weighting of possible sources of evidence. When we use the word "distance" below, the metric case is the cleanest model, not a claim that every useful local relation must be a metric.

The lesson is general: *nearness is modelled, not discovered automatically by the word "data."*

2.5 The k-nearest-neighbour completion operator

We can now define the first explicit completion operator of the book.

Let

$$\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n$$

be a set of fully observed training examples and let x_* be a query whose target is masked.

Definition 2.5 (k-nearest-neighbour completion) *Given a metric d and an integer $k \geq 1$, the k-nearest-neighbour completion completion operator:*

1. *computes $d(x_*, x_i)$ for the training samples;*
2. *selects the indices $N_k(x_*)$ of the k closest samples;*
3. *aggregates the observed targets $\{y_i : i \in N_k(x_*)\}$ according to the target type.*

The learning/use distinction of Chapter 1 is visible even in this simple method. A basic k-NN implementation has little parametric fitting, but the learning stage is not literally empty: the reference sample is stored, preprocessing statistics may be fitted, and choices such as the metric and k are selected using training and validation data. At use time these choices are held fixed while the query is completed:

observe query $\longrightarrow$ compute geometric relations $\longrightarrow$ select local support $\longrightarrow$ aggregate.

This is the first chapter in which the separation between geometry and completion rule is fully operational: geometry determines the local evidence; aggregation determines the readout.

2.5.1 Regression

For scalar regression, the simplest rule is the local mean

$$\widehat{y}_* = \frac{1}{k} \sum_{i \in N_k(x_*)} y_i.$$

For vector-valued targets, the same formula applies componentwise.

This is a piecewise-constant estimator. As x_* moves through sample space, the prediction changes only when the membership of $N_k(x_*)$ changes.

A distance-weighted version is

$$\widehat{y}_* = \frac{\sum_{i \in N_k(x_*)} w_i(x_*) y_i}{\sum_{i \in N_k(x_*)} w_i(x_*)},$$

with, for example,

$$w_i(x_*) = \frac{1}{(d(x_i, x_*) + \delta)^q}, \qquad q > 0,$$

where $\delta > 0$ prevents division by zero. Nearby neighbors then contribute more strongly than distant ones.

2.5.2 Classification

For class labels

$$y_i \in \{1, \dots, C\},$$

local completion may be expressed through empirical class support

$$\widehat{p}(c \mid x_*) = \frac{1}{k} \sum_{i \in N_k(x_*)} \mathbf{1}[y_i = c].$$

The predicted class is

$$\widehat{y}_* = \arg\max_c \widehat{p}(c \mid x_*).$$

This formulation is preferable to thinking only in terms of "majority vote." It exposes the same weighted-completion structure used in regression. The target representation changes, but the geometry and local protocol do not.

With distance weights,

$$\widehat{p}(c \mid x_*) = \frac{\sum_{i \in N_k(x_*)} w_i(x_*) \mathbf{1}[y_i = c]}{\sum_{i \in N_k(x_*)} w_i(x_*)}.$$

The class estimate is again the maximizer.

Principle 2.6 (Geometry and readout) *The same local geometry can support different completion tasks. Classification, regression, and imputation may use the same neighborhood construction while changing only the target representation and aggregation rule.*

2.6 One local rule, several observation patterns

The local mechanism becomes especially useful when we stop identifying k-NN exclusively with label prediction. This chapter concentrates on the special case in which partial observation is created by a mask.

Let a row of a data matrix be

$$x_i = (x_{i1}, \dots, x_{id}).$$

Suppose coordinate j is hidden for a query row x_*. We can select neighbours using only the observed coordinates and complete the missing entry by

$$\widehat{x}_{*j} = \frac{1}{k} \sum_{i \in N_k(x_*; M)} x_{ij}.$$

The notation $N_k(x_*; M)$ emphasizes that the relation between the query and each reference sample is computed from what is actually observed.

If

$$O_* = \{r : M_r = 1\}$$

is the set of observed coordinates of the query, one simple masked Euclidean distance is

$$d_M(x_*, x_i)^2 = \frac{d}{|O_*|} \sum_{r \in O_*} (x_{*r} - x_{ir})^2,$$

provided those coordinates are also available for x_i. The factor $d/|O_*|$ rescales the squared distance for the number of visible coordinates; it does not recover information that the mask has removed.

The procedure is now an imputer rather than a classifier, but its anatomy is unchanged:

$$\begin{aligned} \text{observed coordinates} &\longrightarrow \text{relations on those coordinates} \longrightarrow \text{neighbours} \\ &\longrightarrow \text{completion of the hidden coordinate.} \end{aligned}$$

Example 2.7 (Label masking versus feature masking) *Take the represented object*

$$\widetilde{x} = (x_1, x_2, x_3, y).$$

If only y is hidden, local completion acts as a predictor. If x_2 is hidden and y is available, it acts as a feature imputer. If both x_2 and y are hidden, the same local rule can in principle be used for both readouts, but the relations themselves are now computed from less information.

This last point is important. The mask does not magically "change the true geometry" of the underlying objects. It changes the representation currently available to the learner, and therefore the *effective geometry that can be computed from the observation.* Two samples distinguishable in the full representation may become indistinguishable after informative coordinates are hidden.

The distinction also marks the limit of the mask formalism. If a query is noisy rather than missing coordinates,

$$Y = X + \varepsilon,$$

the observation operator has changed even though $M = \mathbf{1}$. A local method can still be used, but its relation rule may need to account for the noise. The chapter therefore uses masks as a transparent example of partial observation, not as a universal description of every local-learning problem.

2.7 What does k control? Bias, variance, and scale

The integer k is often introduced as a hyperparameter to be tuned by cross-validation. That is correct operationally but incomplete conceptually. In a local completion model, k controls the scale at which the geometry is read.

For small k, completion uses very local evidence. The estimator can follow fine structure, but it is sensitive to noise and sample irregularity. For large k, more evidence is pooled. The estimate is more stable but averages across a larger region and can blur genuine local structure.

This is the familiar bias–variance tradeoff expressed geometrically:

	small k	large k
spatial scale	small	large
adaptivity	high	low
variance	high	lower
bias	lower locally	higher locally

For $k = 1$, each query inherits the target of its nearest training sample. The decision surface for classification becomes a boundary between Voronoi cells. For $k = n$, the method ignores the query completely and returns the global mean or global class mode. Thus the global baseline of section 2.2 is the limiting case in which the neighborhood expands to include the whole dataset.

This gives k a direct interpretation:

$$k \quad \text{controls how much geometry is treated as local.}$$

2.7.1 Cross-validation as scale selection

A validation procedure chooses k by estimating how well each neighbourhood scale completes held-out targets. This belongs to the *learning* stage: once k has been selected, it must not be retuned on the test queries whose performance is being reported.

Geometrically, validation selects a scale of regularity supported by the available data. A small selected k says that useful target variation can be resolved from fine local evidence; a larger k says that broader pooling generalizes better. This interpretation should not be read too literally—the selected value also reflects sample size, noise, metric quality, and the validation criterion—but it is more informative than treating k as an arbitrary knob.

Later models will learn several scales or make scale depend on the current object rather than choosing one global neighbourhood size.

2.8 Kernels: from hard neighbours to soft influence

The set $N_k(x_*)$ creates a hard boundary: points inside the neighborhood influence the completion, while points just outside it do not. A smoother alternative is to allow every sample to contribute with a weight that decays with distance.

Let

$$K : [0, \infty) \to [0, \infty)$$

be a nonnegative weighting profile and let $h > 0$ be a scale parameter. Define

$$w_i(x_*) = K\left(\frac{d(x_i, x_*)}{h}\right).$$

Then the local regression estimate is

$$\widehat{y}_* = \frac{\sum_{i=1}^n K\left(\frac{d(x_i,x_*)}{h}\right) y_i}{\sum_{i=1}^n K\left(\frac{d(x_i,x_*)}{h}\right)}.$$

Definition 2.8 (Kernel neighborhood) *A kernel neighborhood assigns graded influence to observed samples as a function of their relation to the query rather than selecting a hard set of neighbors.*

A common choice is the Gaussian profile

$$K(r) = \exp\left(-\frac{r^2}{2}\right),$$

which yields

$$w_i(x_*) = \exp\left(-\frac{d(x_i, x_*)^2}{2h^2}\right).$$

In the geometry notation of Chapter 1, the weight itself can now serve as the relation descriptor,

$$W(x_*, x_i) = w_i(x_*).$$

Thus the hard distinction "neighbour / non-neighbour" has been replaced by a graded relation. The parameter h is the bandwidth. Small h produces sharply local influence; large h spreads influence over a wider region.

The analogy with k is immediate:

$$\text{small } k \;\leftrightarrow\; \text{small } h, \qquad \text{large } k \;\leftrightarrow\; \text{large } h.$$

Both control the geometric scale of completion, but one does so through neighbor count and the other through a continuous decay profile.

2.8.1 Classification with soft support

For classification, define class support

$$S_c(x_*) = \sum_{i=1}^n w_i(x_*)\mathbf{1}[y_i = c].$$

Normalizing gives

$$\widehat{p}(c \mid x_*) = \frac{S_c(x_*)}{\sum_{r=1}^C S_r(x_*)}.$$

Again the distinction between classification and regression occurs mainly in the readout. The underlying geometry generates a field of local influence around the query.

2.8.2 A note on the word kernel

The word *kernel* is used in several related but not identical ways in machine learning. In local smoothing, a kernel is a distance-dependent weighting function such as the Gaussian profile above. In kernel methods studied later, a positive-semidefinite function

$$k(x, z) = \langle \phi(x), \phi(z) \rangle$$

defines an inner-product geometry in a feature space. The two uses are connected through similarity, but they should not be conflated. Here our emphasis is on kernels as soft neighborhoods.

2.9 Local support is more than a prediction

A classical implementation returns only $\widehat{y}_*$. The completion viewpoint suggests that the neighborhood itself contains useful diagnostics.

Three quantities are especially informative.

2.9.1 Neighbourhood radius

Define

$$r_k(x_*) = d_{(k)}(x_*).$$

A large radius means that the method had to search far in order to collect k examples. This is evidence that the query lies in a sparse region of the observed sample geometry.

Thus two predictions with the same majority vote need not have the same support. A vote of 5 to 2 among very close samples is geometrically different from the same vote among distant samples.

2.9.2 Class or target disagreement

For classification, the local class entropy

$$H_k(x_*) = -\sum_{c=1}^{C} \widehat{p}(c \mid x_*) \log \widehat{p}(c \mid x_*)$$

measures disagreement among neighboring labels.

For regression, the local target variance

$$V_k(x_*) = \frac{1}{k} \sum_{i \in N_k(x_*)} (y_i - \widehat{y}_*)^2$$

plays an analogous role.

These quantities are not complete measures of predictive uncertainty, but they reveal whether the local evidence is coherent.

2.9.3 Total kernel support

For soft neighborhoods, define

$$S(x_*) = \sum_{i=1}^{n} w_i(x_*).$$

A low value indicates that the query receives little support at the chosen bandwidth. The normalized prediction may still look numerically confident, but the unnormalized support exposes that the confidence is built from little local evidence.

This distinction will matter later when we separate

which completion is favoured

from

how much local evidence supports it.

For now the point is simpler: the predicted value and the evidence behind it are different outputs of the same local computation.

Principle 2.9 (Prediction and support are different readouts) *A local learner should be viewed as producing at least two kinds of information: a completion and a description of the local evidence supporting that completion.*

2.10 A worked example

Consider the two-dimensional training data

x_i	y_i
$(0.0, 0.0)$	0
$(0.2, 0.1)$	0
$(0.1, 0.3)$	0
$(1.0, 1.0)$	1
$(1.1, 0.9)$	1
$(0.9, 1.2)$	1

and the query

$$x_* = (0.80, 0.85).$$

Using Euclidean distance, the six query–sample relations have descriptors

i	1	2	3	4	5	6
$d(x_*, x_i)$	1.167	0.960	0.890	0.250	0.304	0.364

to three decimals. For $k = 3$,

$$N_3(x_*) = \{4, 5, 6\},$$

and all three observed labels are 1. Therefore

$$\widehat{p}(1 \mid x_*) = 1, \qquad \boxed{\widehat{y}_* = 1}.$$

The neighbourhood radius is

$$r_3(x_*) = 0.364,$$

and the local class entropy is zero. Thus the prediction, the spatial support, and the label agreement are all visible separately.

Now consider

$$x_{**} = (0.52, 0.52).$$

Its distances are approximately

$$(0.735,\ 0.528,\ 0.474,\ 0.679,\ 0.693,\ 0.779).$$

The three nearest samples are $3, 2, 4$, with labels

$$(0, 0, 1).$$

Hence

$$\widehat{p}(1 \mid x_{**}) = \frac{1}{3}, \qquad \widehat{y}_{**} = 0,$$

but now

$$r_3(x_{**}) \approx 0.679$$

and the binary label entropy is

$$H_3(x_{**}) = -\frac{2}{3}\log\frac{2}{3} - \frac{1}{3}\log\frac{1}{3} \approx 0.637.$$

The second query therefore produces a class prediction just as the first one does, but the evidence has a larger spatial radius and substantial disagreement. These are not cosmetic diagnostics: they tell us that the two completions are supported in very different ways.

2.10.1 Changing the representation changes the induced geometry

Suppose the first coordinate is rescaled by a factor of ten:

$$T(x_1, x_2) = (10x_1, x_2).$$

If Euclidean distance is retained as the relation rule, then

$$d_T(x, z)^2 = 100(x_1 - z_1)^2 + (x_2 - z_2)^2.$$

The underlying observations and labels have not changed, but their representation has. Consequently, Euclidean comparison induces a different geometry.

Using the same training data as above, consider

$$x^\dagger = (0.10, 1.00).$$

In the original representation its distances from the six training samples are approximately

$$(1.005,\ 0.906,\ 0.700,\ 0.900,\ 1.005,\ 0.825).$$

Thus

$$N_3(x^\dagger) = \{3, 6, 4\},$$

whose labels are

$$(0, 1, 1).$$

Hence

$$\widehat{y}^{\dagger} = 1.$$

After applying T, the corresponding distances become approximately

$$(1.414,\ 1.345,\ 0.700,\ 9.000,\ 10.000,\ 8.002).$$

The three nearest neighbours are now

$$N_3^T(x^{\dagger}) = \{3, 2, 1\},$$

with labels

$$(0, 0, 0),$$

so

$$\widehat{y}_T^{\dagger} = 0.$$

Thus nothing about the underlying observations or labels has changed. What changed is the representation in which Euclidean distance is computed. That representation change altered the induced geometry, which altered the neighbourhood, and in this case changed the completion itself:

representation change $\longrightarrow$ geometry change $\longrightarrow$ neighbourhood change $\longrightarrow$ prediction change.

2.11 Decision regions and Voronoi structure

The geometry of 1-NN can be described exactly through a Voronoi partition. For each training sample x_i, define its cell

$$V_i = \{x : d(x, x_i) \leq d(x, x_j) \text{ for all } j\}.$$

Every query in V_i receives the target y_i. Thus the training samples induce a tessellation of the input space, and completion is constant inside each cell.

For classification, decision boundaries are subsets of boundaries between cells carrying different labels. Increasing k replaces this sharp single-cell inheritance with voting over overlapping local configurations, smoothing the decision surface.

This provides an important geometric perspective on model complexity. A 1-NN classifier stores an extremely detailed partition whose complexity grows with the training sample. Increasing k smooths over that partition without learning a low-dimensional parametric decision surface.

Nearest-neighbour methods are therefore examples of *memory-based* completion: the geometry of the training sample remains directly present at prediction time.

2.12 Why local completion can work

The statistical intuition behind local methods is continuity. If the target mechanism varies smoothly in the chosen geometry, then nearby samples should carry information about one another.

For regression, suppose

$$y = m(x) + \varepsilon, \qquad \mathbb{E}[\varepsilon \mid x] = 0,$$

where

$$m(x) = \mathbb{E}[y \mid x]$$

is the regression function. If m is smooth, then for x_i near x_*,

$$m(x_i) \approx m(x_*).$$

Averaging neighboring targets therefore estimates $m(x_*)$.

For classification, the analogous object is the posterior class probability

$$\eta_c(x) = P(y = c \mid x).$$

If η_c changes smoothly in the chosen geometry, local class frequencies approximate the posterior probabilities near the query.

The key phrase is *in the chosen geometry*. Smoothness in one representation need not imply smoothness in another. A target may be highly irregular in the raw coordinates but simple after an appropriate transformation. This observation points directly toward representation learning: if local completion fails because the geometry is wrong, one remedy is to learn a better geometry.

Principle 2.10 (Local smoothness assumption) *Local completion is justified when the target or missing structure varies slowly among objects that the chosen geometry declares to be nearby.*

2.13 The curse of dimensionality

Local methods are conceptually simple, but their central assumption becomes difficult to exploit in high-dimensional spaces.

Consider a unit hypercube $[0,1]^d$. A ball or hypercube containing a fixed fraction of the volume must extend increasingly far along each coordinate as d grows. For an axis-aligned hypercube with side length ℓ, the volume is

$$\ell^d.$$

To capture a fraction q of the total volume, one needs

$$\ell = q^{1/d}.$$

For fixed $q \in (0,1)$,

$$q^{1/d} \to 1 \qquad \text{as } d \to \infty.$$

Thus a region containing a non-negligible fraction of the sample becomes geometrically large in each coordinate.

This is one aspect of the curse of dimensionality. Local evidence becomes sparse because the volume of the ambient space grows much faster than the available sample.

2.13.1 Distance concentration

A related effect is distance concentration. In many high-dimensional distributions, distances between random points become similar relative to their mean. The contrast between "near" and "far" shrinks.

When

$$\frac{d_{\max} - d_{\min}}{d_{\min}}$$

becomes small, nearest neighbors may no longer be meaningfully near. A method can still return the closest k points, but the rank ordering may carry little semantic information.

This is a crucial distinction:

$$\text{having nearest neighbors} \neq \text{having informative local geometry.}$$

Every finite dataset has nearest neighbors. The question is whether nearness corresponds to relevant similarity for the completion task.

2.13.2 Irrelevant coordinates

Suppose only $r \ll d$ features carry information about the target, while the remaining coordinates are independent noise. Euclidean distance accumulates variation over all d coordinates. The irrelevant dimensions can dominate the distance and obscure the informative subspace.

This failure is not primarily a defect of averaging. It is a defect of geometry. The representation makes unrelated objects appear distant and can make genuinely similar objects appear no closer than arbitrary ones.

Three broad remedies will occupy much of the rest of the book:

1. *fit a structured global geometry*, as in linear models;
2. *reduce to a lower-dimensional latent geometry*, as in PCA and factorization;
3. *learn a representation in which useful locality emerges*, as in neural representation learning.

Local completion therefore gives us both a baseline and a diagnosis of what more advanced learning must accomplish.

2.14 When locality fails for reasons other than dimension

High dimension is not the only problem. Local methods can fail even in low-dimensional spaces if the metric does not align with the task.

2.14.1 Nonstationary relevance

A single global metric may be appropriate in one region and inappropriate in another. Feature x_1 may determine similarity near one class boundary while x_2 matters elsewhere. Fixed Euclidean geometry cannot express this without transformation.

2.14.2 Different local scales

Some regions of the data may require small neighborhoods while others require large ones. A single global k or bandwidth h imposes one bias–variance compromise everywhere.

This motivates adaptive locality: allowing neighborhood scale, metric, or weighting to depend on position in the data space.

2.14.3 Disconnected but equivalent objects

Two observations may be far in the raw coordinates but equivalent for the task. Images of the same object under different illumination, sentences with similar meaning but different words, or two states related by a symmetry can be separated by large raw distance. A useful representation should bring task-equivalent objects closer.

2.14.4 Ambiguous local evidence

A query may lie in a region where multiple outcomes genuinely coexist. No change of metric can force a deterministic answer without discarding real uncertainty. Here the proper response is not necessarily a better point estimate but an explicit representation of ambiguity. This is one reason probabilistic and regulated completion will be needed later.

2.15 Computation and memory

Nearest-neighbour methods have almost no conventional training cost: they store the training sample. Prediction, however, can be expensive. A naive query requires computing n distances in d dimensions,

$$O(nd),$$

followed by neighbor selection.

Data structures such as k-d trees, ball trees, vantage-point trees, locality-sensitive hashing, and approximate-nearest-neighbour indexes can reduce practical query cost in suitable geometries. Their effectiveness, however, is also dimension dependent. Tree-based exact search often degrades toward brute-force behavior in high dimension.

This computational observation mirrors the statistical one: local geometry is easiest to exploit when the data occupy a space in which neighborhoods are meaningful and efficiently searchable.

There is also a memory tradeoff. Parametric models compress the training sample into a finite parameter vector. Basic k-NN retains the sample itself. It therefore postpones much of the learning work until query time.

This motivates the traditional term *lazy learning*. More precisely, the sample objects are retained so that their geometry relative to each new query can be reconstructed at use time. The training set stores the reference objects; the metric and neighbourhood rule say how relations among those objects are to be computed.

Later methods will compress the training experience into fitted parameters or transform the representation so that useful relations can be recovered more efficiently.

2.16 A practical local-completion checklist

A local method should not be specified only by saying "use k-NN." The result depends on several choices that Chapter 1 taught us to keep separate. For a reproducible experiment, record at least:

1. the underlying object and the quantity to be completed;
2. the observation pattern or observation operator;
3. the representation and any preprocessing fitted on the training data;
4. the relation descriptor—for example Euclidean distance, Hamming distance, or a similarity;
5. the neighbourhood or weighting rule, such as k, ε, or bandwidth h;
6. the completion readout;
7. useful support diagnostics such as neighbourhood radius, total kernel weight, or local disagreement;
8. held-out performance as the relevant choices vary.

This exposes the computational chain

$$\text{observation} \to \text{representation} \to \text{relations} \to \text{local support} \to \text{completion}.$$

A useful diagnostic plot is error versus neighbourhood radius. If errors concentrate where $r_k(x)$ is large, the main problem may be lack of nearby evidence. If errors occur at small radius but high label entropy, local outcomes disagree. If errors occur at small radius and low entropy, the relation rule may be systematically inappropriate, the representation may hide the relevant structure, or the labels may be noisy.

The point is pedagogical as well as practical: once these quantities are exposed, a wrong prediction can often be analysed in terms of a failed modelling assumption rather than treated as an opaque mistake of "the algorithm."

2.17 Practical experiment: one dataset, three masked tasks

A useful exercise is to hold the reference data and local relation rule fixed while changing only which component is hidden. Let the represented object be

$$\widetilde{x} = (x_1, \ldots, x_d, y).$$

Task A: label completion

Hide y on held-out rows. Use the visible features to find neighbours and complete the label. Report classification accuracy together with neighbourhood radius and class entropy.

Task B: feature completion

Hide one numerical feature on held-out rows. Compute relations on the remaining visible features and complete the missing feature by neighbour averaging. Report mean absolute or squared error together with neighbourhood radius.

Task C: mixed completion

Hide both one feature and the label. Compute relations only from the remaining visible coordinates. Use the resulting neighbourhood to complete the feature and the label separately. Compare the results with Tasks A and B.

The purpose is not to obtain a state-of-the-art score. It is to see directly that the observation pattern changes both the quantity to be completed and, because fewer coordinates are available, the effective geometry seen by the local method.

For this particular exercise a simple implementation may deliberately keep learning and use separate:

```
fit(reference_data, metric, scale)
complete(partial_object, mask)
    -> completion, diagnostics
```

Here `fit` stores or prepares the reference structure using training data, whereas `complete` applies the fixed local rule to a new masked observation. This is a teaching interface for the present chapter, not a claim that all learning systems must expose exactly these two software calls.

2.18 From fixed geometry to fitted geometry

Local completion has now forced several issues into the open.

First, there is no meaningful notion of neighbor without a representation and metric. Second, scale matters: very small neighborhoods are unstable, while large neighborhoods erase local structure. Third, high-dimensional raw geometry is often poor. Fourth, a prediction should be distinguished from the strength and coherence of the evidence supporting it.

The most important limitation is that the relation rule used by basic k-NN is largely chosen before target fitting. Standardization constructs a representation from training statistics, and k may be selected by validation, but the directions in feature space that matter for the target are not explicitly fitted by the predictor.

This suggests the next step.

Rather than asking

which training samples are close to this query?

we can learn a global direction or surface that determines how features combine to complete the target.

For regression this leads to models of the form

$$\widehat{y} = w^\top x + b.$$

For classification it leads to separating scores and probabilities built from the same fitted feature geometry.

The transition from local to linear completion is therefore not merely a change of algorithm. It is a change in where the structure resides:

local completion	:	reference samples retained; query relations computed locally,
linear completion	:	training experience summarized by fitted feature directions.

The next chapter develops that transition.

2.19 Summary

The main conclusions of this chapter are:

1. Local completion uses geometrically related observed sample objects to estimate an unresolved target or feature.
2. Representation and relation rule are distinct modelling choices: together they induce the neighbourhood structure on which the method relies.
3. k-NN classification, regression, and feature imputation can share the same local relation rule while differing in the observation pattern and readout.
4. The neighborhood size k and kernel bandwidth h are geometric scale parameters controlling the bias–variance tradeoff.
5. Kernels replace hard neighborhoods by graded fields of local influence.
6. Neighborhood radius, target disagreement, and total support are useful diagnostics distinct from the final prediction.
7. High dimension can destroy meaningful locality through sparsity, irrelevant coordinates, and distance concentration.
8. When fixed local relations are inadequate, the next move is to fit task-relevant structure or learn a representation that induces a better geometry.

Local completion is therefore more than an elementary benchmark. It is the first method for which the completion viewpoint pays for itself pedagogically: we can point to the observed query, the sample objects, the relations among them, the local evidence selected by those relations, and the readout that produces the answer. The next chapters will ask how much of that explicit sample geometry can be replaced by structure fitted from data.

Chapter 3

Linear Completion: Fitting Global Directions

3.1 From neighborhoods to directions

The previous chapter used a local answer to the completion problem:

> *To complete a missing value, look at observed objects that are nearby.*

That answer makes the training sample itself carry much of the geometry. A query is completed from the targets attached to nearby rows, and the main modelling choices are the representation, the metric, and the neighborhood scale.

Linear methods make a different commitment. They do not ask which individual training samples should be consulted for each query. Instead, they fit a small number of global directions in feature space and use those directions to constrain every completion.

For a scalar target, we represent both the sample vector and the fitted direction as row vectors. The basic model is

$$\widehat{y} = xw^\top + b, \qquad x, w \in \mathbb{R}^{1\times d}, \quad b \in \mathbb{R}.$$

The vector w assigns a signed influence to each feature and defines a direction through the feature space. Once w and b have been learned, the training sample no longer needs to be searched at prediction time. Its relevant structure has been compressed into the fitted parameters.

This is the central transition of the chapter:

local completion	:	query-dependent support from nearby samples,
linear completion	:	global support through fitted feature directions.

The word *global* matters. A linear rule uses the same direction w everywhere in the input space. This is both its strength and its limitation. The model is compact, interpretable, and statistically efficient when the target varies approximately along a small number of global directions. It fails when the required relation bends, changes regime, or depends strongly on local context.

Definition 3.1 (Linear completion) *A linear completion is a completion rule in which the missing target or component is estimated from an affine function of the observed features,*

$$\widehat{y} = xw^\top + b,$$

or from a monotone or probabilistic readout of one or more such linear scores.

The final clause allows classification to be treated in the same framework. A linear classifier does not usually return the raw value $xw^\top + b$; it interprets that value as a score, a log-odds, a margin, or one component of a vector of class scores. The fitted direction remains the structural core.

3.2 The design matrix and the completion target

Let the observed training data be

$$\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n, \qquad x_i \in \mathbb{R}^{1\times d}.$$

Thus each x_i is a row vector. Collect the sample vectors into the design matrix

$$X = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^{n\times d},$$

and the scalar targets into

$$y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \in \mathbb{R}^n.$$

To keep the notation compact, we will first absorb the intercept into the matrix. Define

$$\widetilde{X} = \begin{bmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \in \mathbb{R}^{n\times(d+1)}$$

and

$$\beta = \begin{bmatrix} b \\ w^\top \end{bmatrix} \in \mathbb{R}^{d+1}.$$

Then the vector of fitted values is

$$\widehat{y} = \widetilde{X}\beta.$$

In the least-squares derivations below, we use $X \in \mathbb{R}^{n\times p}$ for the matrix actually used in fitting and $\beta \in \mathbb{R}^p$ for its parameter column. Thus $p = d$ and $\beta = w^\top$ when no intercept is included, while $p = d + 1$, $X = \widetilde{X}$, and $\beta = [b; w^\top]$ when the intercept is absorbed into the design matrix. This convention lets us write $X\beta$ without ambiguity.

In the label-augmented viewpoint of chapter 1, the training object is

$$\widetilde{Z} = [X \mid y].$$

At prediction time a new row is observed as

$$(x_*, ?),$$

and the missing target is completed by

$$(x_*, ?) \longmapsto (x_*, x_* w^\top + b).$$

Nothing about this formulation requires us to think of the target as ontologically different from the features. The distinction is operational: the mask says which component is hidden and which components are available to constrain it.

3.2.1 Multiple outputs

For a vector target represented as a row $y_i \in \mathbb{R}^{1\times q}$, collect the targets into

$$Y \in \mathbb{R}^{n\times q}.$$

A linear completion becomes

$$\widehat{Y} = XW, \qquad W \in \mathbb{R}^{d\times q}.$$

Each column of W defines a fitted direction for one output coordinate. The same geometry therefore extends naturally from scalar regression to multi-output regression and to the linear output layers used throughout modern neural networks.

3.3 Least squares as geometric completion

The classical least-squares problem chooses parameters that minimize squared completion error on the observed training targets:

$$\widehat{\beta} = \arg\min_{\beta} \|y - X\beta\|_2^2.$$

The vector

$$r = y - X\widehat{\beta}$$

is the ordinary regression residual: the part of the target vector not explained by the fitted linear completion.

Since the fitted vector $X\widehat{\beta}$ need not equal y, least squares does not solve $X\beta = y$ directly. A change $\delta\beta$ in the parameters moves the fitted vector by $X\,\delta\beta$, hence only along directions inside the column space of X. At the minimum, no such allowed motion can shorten the residual. If the residual had a nonzero component along one of these directions, moving the fit slightly along that component would decrease $\|r\|_2$. Therefore the minimizing residual must be orthogonal to every column of X:

$$r = y - X\widehat{\beta}, \qquad X^\top r = 0.$$

Hence

$$X^\top (y - X\widehat{\beta}) = 0,$$

and rearranging gives

$$\boxed{X^\top X\widehat{\beta} = X^\top y,}$$

the normal equations of least squares.

If $X^\top X$ is invertible,

$$\widehat{\beta} = (X^\top X)^{-1}X^\top y.$$

If $X^\top X$ is singular, a least-squares solution still exists, although the coefficient vector need not be unique. The Moore–Penrose pseudoinverse selects the minimum-norm solution:

$$\widehat{\beta} = X^+ y = (X^\top X)^+ X^\top y.$$

The formula is useful, but the geometry is more revealing than the algebra.

3.3.1 Projection onto the column space

The set of all fitted vectors obtainable from a linear model is

$$\mathcal{C}(X) = \{X\beta : \beta \in \mathbb{R}^p\},$$

the column space of the design matrix. Least squares selects the point in this subspace closest to y in Euclidean norm:

$$\widehat{y} = \operatorname*{arg\,min}_{z \in \mathcal{C}(X)} \|y - z\|_2^2.$$

Thus

$$\widehat{y} = P_X y,$$

where, for full column rank,

$$P_X = X(X^\top X)^{-1}X^\top$$

is the orthogonal projection matrix onto $\mathcal{C}(X)$. More generally, including the rank-deficient case,

$$P_X = XX^+.$$

The residual is orthogonal to every fitted direction:

$$X^\top r = 0.$$

Equivalently,

$$\langle r, Xv\rangle = 0 \qquad \text{for every } v \in \mathbb{R}^p.$$

The learner has decomposed the observed target into

$$y = \widehat{y} + r, \qquad \widehat{y} \in \mathcal{C}(X), \quad r \perp \mathcal{C}(X).$$

Principle 3.2 (Least squares as projection) *Least squares completes targets by projecting them onto the subspace of target patterns expressible as linear combinations of the observed feature columns.*

This statement connects the sample and feature views of the matrix. In the sample view, the fitted rule predicts each row. In the feature view, the columns of X span the family of target patterns the model is capable of expressing across the training sample.

3.3.2 What exactly is being learned?

It is tempting to say that linear regression "learns a line." In one dimension that is literally true, but the statement hides the more general structure. The model learns:

1. a direction w in feature space;
2. a scalar coordinate $xw^\top$ for every sample;
3. an affine rule that turns that coordinate into a target completion.

The scalar

$$s(x) = xw^\top + b$$

is a linear score. For regression, the score itself is the completion. For classification, the same object will be passed to a discrete or probabilistic readout.

The learned direction therefore acts as a task-dependent compression:

$$x \in \mathbb{R}^d \quad \longmapsto \quad s(x) \in \mathbb{R}.$$

All variations in x orthogonal to w are invisible to this scalar prediction. This is one reason the linear model is interpretable: it declares explicitly which direction of feature variation matters for the target.

3.4 Feature geometry inside least squares

The matrix

$$X^\top X$$

contains the inner products among feature columns. If the columns have been centered, then

$$\frac{1}{n} X^\top X$$

is the empirical covariance matrix up to the usual normalization convention.

The least-squares solution therefore depends not only on the association of each feature with the target but also on the relations among the features themselves.

To see this, write the normal equations as

$$G\widehat{\beta} = c,$$

where

$$G = X^\top X, \qquad c = X^\top y.$$

The matrix G is the Gram matrix of feature columns. The vector c records their alignment with the target. Solving for $\widehat{\beta}$ balances target alignment against redundancy among features.

This is the first place in the book where the feature geometry itself directly controls fitting.

3.4.1 Orthogonal features

If the centered feature columns are orthogonal and normalized so that

$$X^\top X = I,$$

then

$$\widehat{\beta} = X^\top y.$$

Each coefficient is simply the alignment of one feature with the target. The features can be treated independently.

3.4.2 Correlated features

If two columns are nearly parallel, then many coefficient combinations can produce similar fitted values. Suppose

$$x_{\cdot 2} \approx x_{\cdot 1}.$$

Then increasing one coefficient while decreasing the other may leave

$$X\beta$$

almost unchanged. The target completion can be stable while the coefficients themselves become unstable.

This is the phenomenon of multicollinearity. Geometrically, the feature directions do not span the target-pattern space cleanly. Algebraically, $X^\top X$ becomes ill-conditioned.

The issue is important because it separates two questions that are often conflated:

1. Is the predicted completion stable?
2. Are the fitted coefficients individually stable and interpretable?

The answers need not be the same.

3.5 Conditioning and sensitivity

A matrix is ill-conditioning when small perturbations of the data can cause large perturbations of the solution. For least squares, the singular values of X make this precise.

Let

$$X = U\Sigma V^\top$$

be a singular value decomposition, with singular values

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0.$$

If X has full column rank,

$$\widehat{\beta} = V\Sigma^{-1}U^\top y.$$

Components aligned with a small singular value are multiplied by

$$\frac{1}{\sigma_j}.$$

Noise in those directions can therefore be greatly amplified.

The condition number

$$\kappa(X) = \frac{\sigma_{\max}}{\sigma_{\min}}$$

measures the disparity between the best-supported and worst-supported directions. A large value signals that some directions in coefficient space are weakly constrained by the observed data.

This provides a geometric diagnosis of unstable completion. The problem is not simply that "there are too many parameters." It is that the available observations constrain some directions much more strongly than others.

Principle 3.3 (Directional support) *A fitted direction is reliable only to the extent that the observed data constrain variation along that direction.*

This principle will recur in latent models, neural networks, and uncertainty estimation. Weak support is directional, not merely scalar.

3.6 Ridge regression: regulating the fitted direction

Least squares chooses the coefficient vector solely to minimize training residual error. When feature directions are weakly supported, this can produce large coefficients that compensate for one another.

Ridge regression introduces an explicit preference for smaller coefficient vectors:

$$\widehat{\beta}_\lambda = \arg\min_\beta \left\{ \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \right\}, \qquad \lambda \geq 0.$$

Usually the intercept is excluded from the penalty; the notation above assumes centered data or omits that detail for clarity.

The solution is

$$\widehat{\beta}_\lambda = (X^\top X + \lambda I)^{-1} X^\top y.$$

The added term

$$\lambda I$$

changes the geometry of the optimization problem. Directions that are poorly constrained by $X^\top X$ can no longer acquire arbitrarily large coefficients at little cost.

Definition 3.4 (Regularization) *regularization is the introduction of an explicit preference or constraint that restricts the family of completions available to a fitted model, usually in order to improve stability or generalization.*

3.6.1 Ridge in singular-vector coordinates

The stabilizing effect is clearest through the SVD. If

$$X = U \Sigma V^\top,$$

then

$$\widehat{\beta}_\lambda = V \operatorname{diag}\left(\frac{\sigma_j}{\sigma_j^2 + \lambda}\right) U^\top y.$$

Compare this with ordinary least squares,

$$\widehat{\beta} = V \operatorname{diag}\left(\frac{1}{\sigma_j}\right) U^\top y.$$

For large σ_j, the ridge factor

$$\frac{\sigma_j}{\sigma_j^2 + \lambda}$$

is close to $1/\sigma_j$. For small σ_j, it remains bounded instead of exploding. Ridge therefore attenuates precisely the directions in which the data provide weak geometric support.

This is more informative than the slogan "ridge shrinks coefficients." Ridge performs anisotropic stabilization in the singular directions of the design matrix, even though the penalty $\|\beta\|_2^2$ itself is isotropic in coefficient coordinates.

3.6.2 The constrained form

The ridge problem is equivalent, for an appropriate relation between λ and t, to

$$\min_\beta \|y - X\beta\|_2^2 \quad \text{subject to} \quad \|\beta\|_2^2 \le t.$$

The feasible parameter set is an Euclidean ball. We can therefore interpret ridge as saying:

> Among linear completions that fit the observed targets, prefer those using a direction of limited magnitude.

The model no longer asks only what completion best matches the training data. It asks what completion is justified under a prior geometric constraint on the fitted direction.

Principle 3.5 (Regularized completion) *Regularization changes completion by changing the set or relative cost of admissible fitted directions.*

This is the first mild form of regulation in the book. It is not yet the dynamic regulation developed later, but it already separates raw fit from controlled fit.

3.7 Why scaling matters again

Feature scaling mattered for nearest neighbors because it changed distances. It matters for linear models for a different but equally geometric reason: it changes what a coefficient norm means.

Suppose feature x_j is rescaled by a factor a:

$$x_j' = a x_j.$$

To preserve the same prediction, its coefficient must transform as

$$\beta_j' = \frac{\beta_j}{a}.$$

An L_2 penalty then changes from

$$\beta_j^2$$

to

$$\frac{\beta_j^2}{a^2}.$$

Thus ridge penalizes directions differently after rescaling even if the underlying predictions can be represented identically.

Standardization therefore serves two distinct roles across the first two chapters:

- in local completion, it makes distance contributions comparable;
- in regularized linear completion, it makes coefficient penalties comparable.

This is another reason to treat preprocessing as part of the model rather than as an administrative step performed before the "real" learning begins.

3.8 Bias, variance, and the regularization path

The parameter λ controls a bias–variance tradeoff.

At

$$\lambda = 0,$$

ridge reduces to least squares. As λ increases, coefficient magnitudes shrink. Training error usually rises, but sensitivity to noise decreases.

In the limit

$$\lambda \to \infty,$$

the penalized coefficients approach zero, leaving only the unpenalized intercept if one is present. The model approaches a global-mean completion.

Thus λ controls how strongly the learner is allowed to use fitted feature directions:

	small λ	large λ
fit flexibility	high	low
coefficient magnitude	larger	smaller
variance	higher	lower
bias	lower	higher
weak singular directions	less suppressed	more suppressed

A validation procedure selecting λ therefore chooses how much weakly supported directional structure should be trusted.

The sequence

$$\lambda \longmapsto \widehat{\beta}_\lambda$$

is the regularization path. Plotting it can reveal which coefficients remain stable as the model moves from weak to strong regularization.

3.9 Other parameter geometries: sparsity and the L_1 norm

Ridge uses the Euclidean norm

$$\|\beta\|_2^2.$$

A different regularizer produces a different geometry of admissible solutions. The lasso uses

$$\widehat{\beta}_\lambda^{(1)} = \arg\min_\beta \left\{ \|y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right\},$$

where

$$\|\beta\|_1 = \sum_{j=1}^{d} |\beta_j|.$$

The associated constraint set

$$\|\beta\|_1 \le t$$

has corners aligned with coordinate axes. Optima often occur at these corners, making many coefficients exactly zero.

This induces sparsity: completion is forced to rely on a smaller subset of feature directions.

The contrast is important:

	L_2 / ridge	L_1 / lasso
parameter geometry	Euclidean ball	cross-polytope
typical effect	smooth shrinkage	sparse selection
weakly supported directions	attenuated	often removed

We will not develop sparse modelling in full here. Its role in this chapter is conceptual: regularization is not one generic operation. Different regularizers encode different geometries of acceptable completion rules.

3.10 Linear classification: completing a discrete target

Regression returns a real-valued target directly. Classification requires a discrete readout. For binary labels

$$y \in \{-1, +1\},$$

a linear classifier computes the score

$$s(x) = xw^\top + b$$

and predicts

$$\widehat{y} = \begin{cases} +1, & s(x) \ge 0, \\ -1, & s(x) < 0. \end{cases}$$

The set

$$\{x : xw^\top + b = 0\}$$

is a decision hyperplane. It divides the feature space into two half-spaces.

The vector w is perpendicular to the hyperplane. Therefore the classifier again learns a direction. The sign of the projection onto that direction determines the completed class.

The signed Euclidean distance from a point x to the hyperplane is

$$\frac{xw^\top + b}{\|w\|_2}.$$

Thus the score is proportional to geometric distance from the decision boundary when w is fixed.

This is the classification analogue of the regression projection. Both methods compress the observed feature vector to one task-relevant coordinate:

$$x \longmapsto s(x) = xw^\top + b.$$

The difference lies in the readout applied to s.

3.10.1 Classification is still completion

In the augmented matrix

$$\widetilde{Z} = [X \mid y],$$

the label is simply the masked component at prediction time. A linear classifier completes that component by first computing the fitted score and then applying a class readout.

The structural chain is

$$x_* \xrightarrow{xw^\top + b} s_* \xrightarrow{\text{readout}} \widehat{y}_*.$$

This separation between score and readout will be useful. Different losses can fit the same kind of linear score while giving it different interpretations.

3.11 Logistic completion: from score to probability

A hard threshold gives a class but does not provide a probabilistic interpretation. Logistic regression models the conditional class probability through the sigmoid function

$$\sigma(s) = \frac{1}{1 + e^{-s}}.$$

For labels $y \in \{0, 1\}$, define

$$p(y = 1 \mid x) = \sigma(xw^\top + b).$$

Then

$$p(y = 0 \mid x) = 1 - \sigma(xw^\top + b).$$

The linear score is the log-odds:

$$\log \frac{p(y = 1 \mid x)}{p(y = 0 \mid x)} = xw^\top + b.$$

This makes the geometry explicit. Equal increments along the direction w produce equal increments in log-odds, not in probability itself.

Definition 3.6 (Logistic completion) *logistic completion is binary target completion in which a fitted linear score is mapped through the logistic function to a conditional class probability.*

The usual negative log-likelihood, or binary cross-entropy, is

$$\mathcal{L}(w, b) = -\sum_{i=1}^{n} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right],$$

where

$$p_i = \sigma(x_i w^\top + b).$$

Training selects the direction that makes the observed labels probable under this model.

3.11.1 Why not least squares on class labels?

One can encode classes numerically and fit least squares, but doing so imposes an error geometry designed for continuous targets. Logistic regression uses a loss matched to Bernoulli outcomes and ensures that predicted probabilities lie in $[0, 1]$.

This illustrates an important separation:

> The geometry of the predictor and the geometry of the loss are different modelling choices.

The predictor geometry says how features combine into a score. The loss says how discrepancies between score-induced outputs and observed targets are penalized during fitting.

3.12 Multiclass linear completion and softmax

For C classes, assign one score to each class:

$$s_c(x) = x w_c^\top + b_c, \qquad c = 1, \ldots, C.$$

Collect the class directions as columns of

$$W = \begin{bmatrix} w_1^\top & \cdots & w_C^\top \end{bmatrix} \in \mathbb{R}^{d \times C}, \qquad b = \begin{bmatrix} b_1 & \cdots & b_C \end{bmatrix} \in \mathbb{R}^{1 \times C}.$$

Then the row vector of class scores is

$$s(x) = xW + b.$$

The softmax readout converts these scores to probabilities:

$$p(y = c \mid x) = \frac{e^{s_c(x)}}{\sum_{r=1}^{C} e^{s_r(x)}}.$$

The completed class is typically

$$\widehat{y} = \arg\max_c s_c(x) = \arg\max_c p(y = c \mid x).$$

Pairwise class boundaries satisfy

$$s_c(x) = s_r(x),$$

or

$$x(w_c - w_r)^\top + (b_c - b_r) = 0.$$

They are therefore hyperplanes. Multiclass linear classification partitions the feature space into regions determined by comparisons among global fitted directions.

This is a useful geometric contrast with k-NN. A nearest-neighbor classifier can produce highly irregular boundaries following the training sample. A softmax linear classifier can only produce boundaries assembled from hyperplanes.

3.13 Margins: how far from indecision?

A predicted class alone hides how close the query lies to the decision boundary. For binary labels $y_i \in \{-1, +1\}$, define the functional margin

$$m_i = y_i(x_i w^\top + b).$$

The corresponding geometric margin is

$$\gamma_i = \frac{y_i(x_i w^\top + b)}{\|w\|_2}.$$

A positive margin means the sample is correctly classified; a negative margin means it lies on the wrong side of the boundary. The magnitude measures distance from indecision in the fitted linear geometry.

For a query with predicted sign $\widehat{y}$, one can similarly inspect

$$\gamma(x) = \frac{|xw^\top + b|}{\|w\|_2}.$$

A small value means the prediction is geometrically fragile to perturbations normal to the boundary.

Definition 3.7 (Margin) *The margin is a measure of separation between a sample and a decision boundary, usually expressed through the signed or absolute linear score normalized by the size of the weight vector.*

Margin is therefore a diagnostic distinct from the class label, just as neighborhood radius was a diagnostic distinct from the k-NN prediction.

3.13.1 Maximum-margin fitting

For linearly separable data, one may choose among the many separating hyperplanes by maximizing the minimum geometric margin. With an appropriate normalization, the hard-margin support vector machine solves

$$\min_{w,b} \frac{1}{2}\|w\|_2^2$$

subject to

$$y_i(x_i w^\top + b) \geq 1 \qquad \text{for all } i.$$

This is another form of regulated linear completion: the training labels may be fitted by many boundaries, but the model prefers the boundary that separates them with the largest buffer.

For nonseparable data, slack variables or hinge loss permit violations while still penalizing small margins. We will not develop support vector machines as a separate algorithmic branch here; the point is conceptual. Least squares, logistic regression, and maximum-margin classification all fit global linear directions, but they encode different criteria for which direction is preferable.

3.14 Loss geometry and what the model treats as an error

The fitted direction depends on the loss used during training. Three familiar choices illustrate different notions of error.

For regression, squared loss is

$$\ell_{\mathrm{sq}}(y, s) = (y - s)^2.$$

Large residuals are penalized quadratically.

For binary logistic classification with $y \in \{-1, +1\}$, logistic loss can be written

$$\ell_{\log}(y, s) = \log(1 + e^{-ys}).$$

It penalizes examples according to their signed score and decreases smoothly as the margin becomes strongly positive.

The hinge loss is

$$\ell_{\mathrm{hinge}}(y, s) = \max(0, 1 - ys).$$

It assigns zero loss once the functional margin exceeds one.

All three can operate on a linear score, yet they produce different fitted directions because they value errors differently.

This distinction is important for the completion viewpoint. A learning method contains at least three separable structural choices:

representation + predictor geometry + loss / fitting criterion.

Regularization adds a fourth:

admissible parameter geometry.

Much of advanced machine learning consists of changing one of these components while leaving the others recognizable.

3.15 A worked regression example

Consider the centered data

$$X = \begin{bmatrix} -2 & -1 \\ -1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 2 & 0 \end{bmatrix}, \qquad y = \begin{bmatrix} -3 \\ -1 \\ 1 \\ 1 \\ 2 \end{bmatrix}.$$

The two feature columns define two directions in $\mathbb{R}^5$, the space of target patterns across the five observations. Linear regression searches their span for the vector closest to y.

Compute

$$X^\top X = \begin{bmatrix} 10 & 1 \\ 1 & 2 \end{bmatrix}, \qquad X^\top y = \begin{bmatrix} 13 \\ 4 \end{bmatrix}.$$

The least-squares coefficients satisfy

$$\begin{bmatrix} 10 & 1 \\ 1 & 2 \end{bmatrix} \widehat{\beta} = \begin{bmatrix} 13 \\ 4 \end{bmatrix},$$

which gives

$$\widehat{\beta} = \begin{bmatrix} 22/19 \\ 27/19 \end{bmatrix} \approx \begin{bmatrix} 1.158 \\ 1.421 \end{bmatrix}.$$

For a new observation

$$x_* = (1.5, 0.5),$$

the completed target is

$$\widehat{y}_* = x_*^\top \widehat{\beta} \approx 2.447.$$

Now add ridge regularization with $\lambda = 2$:

$$\widehat{\beta}_2 = (X^\top X + 2I)^{-1} X^\top y.$$

Then

$$X^\top X + 2I = \begin{bmatrix} 12 & 1 \\ 1 & 4 \end{bmatrix},$$

so

$$\widehat{\beta}_2 = \frac{1}{47} \begin{bmatrix} 48 \\ 35 \end{bmatrix} \approx \begin{bmatrix} 1.021 \\ 0.745 \end{bmatrix}.$$

The second direction has been shrunk substantially because the regularized criterion assigns a cost to coefficient magnitude as well as residual error.

The example is small, but it exposes the complete anatomy:

$$\text{feature relations} \to \text{fitted direction} \to \text{regularized direction} \to \text{target completion}.$$

3.16 A worked classification example

Suppose a fitted binary classifier has

$$w = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \qquad b = -0.5.$$

For the query

$$x_* = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix},$$

the score is

$$s(x_*) = 2(1) - 1(0.5) - 0.5 = 1.$$

A hard linear classifier predicts the positive class.

A logistic readout instead gives

$$p(y = 1 \mid x_*) = \sigma(1) \approx 0.731.$$

The geometric distance to the decision hyperplane is

$$\frac{|s(x_*)|}{\|w\|_2} = \frac{1}{\sqrt{5}} \approx 0.447.$$

These are three different readouts from the same fitted direction:

1. the discrete completion: positive class;
2. the model probability: approximately 0.731;
3. the geometric boundary distance: approximately 0.447.

They should not be confused. In particular, a logistic probability is not the same thing as geometric support or distance from the training distribution. A point far outside the region occupied by the training data can receive an extreme logistic probability simply because its linear score is large.

This is a first warning that predictive confidence and geometric support are different notions.

3.17 Prediction diagnostics beyond the fitted value

As in local completion, a linear learner should expose more than its final prediction. Least squares is especially useful because the fitted values and the residuals can both be written as explicit linear operators on the observed target vector.

3.17.1 Residual structure

For regression, the training residuals are

$$r_i = y_i - \widehat{y}_i,$$

or, in vector form,

$$r = y - \widehat{y}.$$

Plots of residuals against fitted values or features can reveal curvature, heteroscedasticity, regime changes, and outliers that the fitted global relation cannot represent.

A random-looking residual cloud supports the adequacy of the linear mean structure. Systematic residual patterns indicate that organized target structure remains after the part representable by the current linear model has been removed.

This interpretation is especially important for this book. A residual is not merely “error.” It is the part of the observed target that the current representation and fitted linear relation do not account for.

3.17.2 The hat matrix: how the fitted values are assembled

Assume first that the design matrix

$$X \in \mathbb{R}^{n \times p}$$

has full column rank. Ordinary least squares gives

$$\widehat{\beta} = (X^\top X)^{-1} X^\top y.$$

Substituting this expression into $\widehat{y} = X\widehat{\beta}$ gives

$$\widehat{y} = X(X^\top X)^{-1} X^\top y = Hy,$$

where

$$\boxed{H = X(X^\top X)^{-1} X^\top}$$

is the *hat matrix*. The name is literal: the matrix maps the observed target vector y to its fitted version $\widehat{y}$.

The matrix H depends only on the design X, not on the particular observed values of y. It therefore describes how the chosen representation and linear model structure will combine target values once a target vector is supplied. Componentwise,

$$\widehat{y}_i = \sum_{j=1}^{n} h_{ij} y_j.$$

Thus the i-th row of H specifies how all observed target values contribute to the fitted value at sample i.

Two identities explain the geometry of this operator:

$$H^\top = H, \qquad H^2 = H.$$

Hence H is the orthogonal projector onto the column space of X,

$$\widehat{y} = Hy \in \mathrm{col}(X).$$

The fitted target vector is therefore the component of y that can be represented by the linear subspace spanned by the columns of the design matrix. If X is rank deficient, the same projector is written using the Moore–Penrose pseudoinverse,

$$H = XX^+.$$

3.17.3 The hat matrix as a model-induced mask

The language of masking now returns in a less obvious form. Earlier in the book, a mask usually meant an *observation mask*: some coordinates of an object are supplied to the learner and other coordinates are hidden. The hat matrix is also a mask, but not generally a coordinate mask. It is a *model-induced projection mask*: it passes exactly the target-pattern directions that the chosen linear model can represent and suppresses the directions it cannot represent.

This becomes explicit in coordinates adapted to the fitted subspace. Let

$$X = QR$$

be a thin QR factorization, where the columns of $Q \in \mathbb{R}^{n\times p}$ are orthonormal. Then

$$H = QQ^\top.$$

Extend those columns to an orthonormal basis of sample space,

$$U = [Q\ Q_\perp].$$

In this basis,

$$U^\top HU = \begin{bmatrix} I_p & 0 \\ 0 & 0 \end{bmatrix}, \qquad U^\top (I-H)U = \begin{bmatrix} 0 & 0 \\ 0 & I_{n-p} \end{bmatrix}.$$

Thus if

$$U^\top y = \begin{bmatrix} a \\ b \end{bmatrix},$$

then

$$U^\top Hy = \begin{bmatrix} a \\ 0 \end{bmatrix}, \qquad U^\top (I-H)y = \begin{bmatrix} 0 \\ b \end{bmatrix}.$$

In the model-adapted basis the hat matrix is therefore literally a binary mask: it keeps the representable coordinates and zeros the orthogonal ones.

The distinction from an ordinary missing-data mask is important. A coordinate mask determines what information is made available by the data-acquisition process. The hat matrix does not hide entries of the training target from the fitting procedure. Instead, after the design has fixed a linear hypothesis space, it determines which *directions in sample-indexed target space* can appear in a fitted target vector. The complementary operator $I - H$ exposes exactly the target structure that this model-induced mask removes.

We can summarize the two uses of the word *mask* as

observation mask	:	selects observed coordinates,
model-induced mask H	:	selects representable target directions.

Both are observation operators in a broad mathematical sense, but they arise at different stages and should not be conflated.

This viewpoint also clarifies regularization. If

$$X = U_r \Sigma V^\top$$

is a thin singular-value decomposition, then ordinary least squares has spectral gains equal to one on $\mathrm{col}(X)$ and zero on its orthogonal complement. Ridge regression replaces the unit gains by

$$\frac{\sigma_j^2}{\sigma_j^2 + \lambda}, \qquad 0 < \frac{\sigma_j^2}{\sigma_j^2 + \lambda} < 1.$$

Hence ordinary least squares acts as a *hard projection mask*, whereas ridge acts as a *soft spectral mask*: directions are attenuated according to how strongly the design supports them rather than simply passed unchanged. Chapter 4 will meet the same idea again when latent projection keeps selected directions and suppresses the rest.

Principle 3.8 (The hat matrix as a mask) *For least squares, H is a hard mask in a basis adapted to the model subspace. It preserves exactly the target-pattern component that the linear model can represent; $I - H$ preserves exactly the component that the model cannot explain.*

3.17.4 The residual-maker matrix

The connection between the hat matrix and residuals is exact. Since

$$r = y - Hy,$$

we have

$$\boxed{r = (I - H)y.}$$

The matrix

$$M = I - H$$

is often called the *residual-maker matrix*. It is also symmetric and idempotent,

$$M^\top = M, \qquad M^2 = M,$$

and

$$HM = MH = 0.$$

Thus least squares decomposes the observed target orthogonally as

$$\boxed{y = \underbrace{Hy}_{\text{represented by the model}} + \underbrace{(I - H)y}_{\text{residual}}.}$$

The two components satisfy

$$\widehat{y}^\top r = 0, \qquad X^\top r = 0.$$

The residual is therefore not an arbitrary discrepancy. It is the canonical component of the observed target lying outside the subspace that the current linear representation can express.

Idempotence makes this interpretation particularly clear. Once the representable component has been removed, applying the same residual operator again changes nothing:

$$(I - H)r = r.$$

The residual is already entirely outside the fitted subspace. If an intercept column is included in X, the constant vector belongs to $\mathrm{col}(X)$, and the orthogonality condition also implies

$$\sum_{i=1}^{n} r_i = 0.$$

3.17.5 Leverage: the diagonal of the hat matrix

The diagonal entries

$$h_{ii} = H_{ii}$$

measure leverage. They depend only on the feature configuration, not on the observed target. Geometrically,

$$h_{ii} = e_i^\top H e_i = \|He_i\|_2^2,$$

where e_i is the i-th coordinate vector in sample space. Thus h_{ii} measures how strongly the i-th sample coordinate is represented inside the fitted design subspace.

Equivalently, because

$$\widehat{y}_i = h_{ii} y_i + \sum_{j \neq i} h_{ij} y_j,$$

a large h_{ii} means that observation i's own target value receives a large self-weight in its fitted value. Such a sample occupies an unusual or geometrically extreme position relative to the rest of the design and has the *potential* to influence the fitted coefficients strongly.

For an ordinary full-rank design,

$$0 \leq h_{ii} \leq 1, \qquad \sum_{i=1}^{n} h_{ii} = \operatorname{tr}(H) = p.$$

Hence the average leverage is

$$\frac{p}{n}.$$

Leverage is therefore a relative diagnostic: values substantially above the design average deserve attention.

3.17.6 Why leverage and residual must be read together

High leverage by itself does not imply that a point is badly fitted, and a large residual by itself does not imply that a point strongly changes the fitted model. Influence arises from their interaction.

Under the classical homoscedastic model

$$y = X\beta + \varepsilon, \qquad \operatorname{Var}(\varepsilon) = \sigma^2 I,$$

the residual covariance is

$$\operatorname{Var}(r) = \sigma^2 (I - H),$$

so

$$\operatorname{Var}(r_i) = \sigma^2 (1 - h_{ii}).$$

A high-leverage observation is therefore, in a precise sense, allowed to fit itself more closely. Its raw residual can look modest even when the point is geometrically unusual. This is why standardized or studentized residuals correct for leverage, using a factor proportional to

$$\frac{r_i}{\sqrt{1 - h_{ii}}}.$$

An especially useful identity connects the ordinary residual to leave-one-out prediction. Let $\widehat{y}_{(i)}$ denote the prediction at sample i obtained after fitting the model without observation i. Then

$$\boxed{y_i - \widehat{y}_{(i)} = \frac{r_i}{1 - h_{ii}}.}$$

Thus the same hat matrix that produces the fitted values also tells us how much a training residual would expand when the observation is no longer allowed to participate in its own fit.

Classical influence measures combine these two ingredients explicitly. For example, Cook's distance is proportional to

$$r_i^2 \, \frac{h_{ii}}{(1 - h_{ii})^2}.$$

The message is simple: leverage measures the *opportunity* for influence supplied by the design; the residual measures disagreement with the fitted relation. A point becomes especially important when both are large.

3.17.7 Condition number

The condition number of the design matrix reports whether some fitted directions are weakly determined. It is a global diagnostic of feature geometry rather than an error measure on individual samples.

3.17.8 Margin

For classification, the margin measures proximity to the fitted decision boundary. Small margin indicates local fragility of the discrete readout.

These diagnostics occupy different levels:

diagnostic	what it interrogates
$(I - H)y$	target structure outside the fitted linear subspace
h_{ii}	sample leverage supplied by the design
studentized/LOO residual	disagreement after accounting for leverage
condition number	weakly constrained global directions
margin	proximity to classification boundary

A mature learning system should keep these distinctions visible. The hat matrix makes the separation particularly explicit:

$$\begin{aligned} H &: \text{ hard model mask retaining what the linear model can represent,} \\ I - H &: \text{ complementary mask retaining what remains unexplained.} \end{aligned}$$

Principle 3.9 (Projection, masking, and residual diagnostics) *For least squares, fitted values and residuals are complementary projections determined by the same design. In a basis adapted to the design subspace, they are complementary binary masks. The hat matrix retains the representable target-pattern component, the residual-maker matrix retains the unresolved component, and leverage quantifies how strongly individual observations participate in that projection.*

3.18 When global linear completion works

Linear models are often described as simple baselines. That description is useful operationally but can be misleading conceptually. A linear model is exactly appropriate whenever the task-relevant variation can be represented by stable global directions.

Examples include:

- approximately additive effects;
- local physical regimes already expressed in good coordinates;
- high-dimensional problems with many weak contributions and strong regularization;

- sparse settings in which only a small subset of variables matters;
- final readout layers applied to a representation whose nonlinear structure has already been resolved upstream.

The last case is especially important. Modern deep networks frequently end with a linear or affine map. Their success does not show that linear completion is obsolete. It shows that a learned representation can transform a difficult problem into one that is nearly linear at the readout.

Thus one of the recurring goals of representation learning will be:

make the target simple in the new geometry.

A linearly separable or linearly predictable latent space is often precisely what the earlier nonlinear layers are trained to construct.

3.19 When global linear completion fails

The same global nature that makes linear models compact also creates characteristic failure modes.

3.19.1 Curvature

If the true regression relation is

$$y = x^2 + \varepsilon,$$

a one-dimensional linear rule cannot represent the curvature. Residuals will show a systematic pattern.

The immediate remedy need not be a more complicated learning algorithm. We can change the representation by adding a feature

$$\phi(x) = (x, x^2),$$

a move that makes the problem linear in the new coordinates.

This illustrates a fundamental fact:

Linear versus nonlinear is always relative to a representation.

3.19.2 Interactions

Suppose the target depends on

$$y \approx x_1 x_2.$$

A linear model in (x_1, x_2) cannot express the interaction, but a linear model in

$$\phi(x) = (x_1, x_2, x_1 x_2)$$

can.

Feature engineering therefore modifies the geometry by constructing new directions in which the target becomes easier to complete.

3.19.3 Regime changes

A single direction may be inappropriate when the data obey different relations in different regions:

$$y \approx \begin{cases} xw_1^\top, & x \in R_1, \\ xw_2^\top, & x \in R_2. \end{cases}$$

Local methods can sometimes adapt to this naturally; a global linear model averages across regimes unless regime indicators are represented explicitly.

3.19.4 Extrapolation without support

Linear models extend their fitted rule indefinitely:

$$\widehat{y} = xw^\top + b$$

for every $x \in \mathbb{R}^d$. This makes extrapolation easy computationally but potentially dangerous epistemically. The model may produce a precise number or extreme class probability in regions where no training observations provide support.

Unlike k-NN, which at least exposes an expanding neighborhood radius, a basic linear model has no built-in locality diagnostic. This is one reason later chapters will distinguish model output from support in representation space.

3.20 From raw coordinates to basis expansions

A powerful way to extend linear completion while preserving its fitting machinery is to transform the input through a basis map

$$\phi : \mathbb{R}^d \to \mathbb{R}^p$$

and fit

$$\widehat{y} = \phi(x)w^\top + b.$$

The model is linear in the transformed coordinates even when it is nonlinear in the original input.

Examples include polynomial features, spline bases, radial basis functions, and hand-designed domain features.

This separates two questions:

1. How is the representation $\phi(x)$ constructed?
2. How is the target completed from that representation?

If ϕ is fixed in advance, only the readout direction is learned. If ϕ is learned from data, we move toward representation learning.

This distinction marks a major conceptual bridge in the book:

linear completion in fixed features $\longrightarrow$ learning the features in which completion is linear.

Before taking that bridge, however, we will study another way to transform geometry: discovering latent directions directly from the data matrix.

3.21 Linear completion under arbitrary masks

The standard supervised formulation masks only the target column. But the completion viewpoint asks whether a linear rule can operate when other components are also hidden.

Here *mask* again means an externally specified coordinate-observation pattern. This should be distinguished from the model-induced mask H of section 3.17.3. The former determines what coordinates reach the learner; the latter determines, once a linear design has been chosen, what target-pattern directions the model is capable of reproducing. A problem may therefore contain both kinds of masking at once.

Suppose the full row is

$$z = (z_O, z_H),$$

where z_O contains observed coordinates and z_H hidden coordinates. A general linear completion has the form

$$\widehat{z}_H = Az_O + c.$$

For each mask pattern, the matrix A maps the visible subspace to the hidden subspace.

This formulation includes ordinary regression, multi-output regression, and linear imputation as special cases. If the data are approximately jointly Gaussian, the conditional mean of hidden variables given observed variables is itself affine, making linear completion statistically natural.

However, arbitrary-mask completion exposes a limitation of fitting one supervised coefficient vector. Different masks change which coordinates are inputs and which are outputs. A reusable completion system therefore needs either:

- a family of mask-specific linear maps;
- a joint covariance or latent model from which the appropriate conditional map can be derived;
- or a more flexible architecture that accepts masks explicitly.

This issue prepares the transition to matrix factorization, probabilistic modelling, and the Completion Agent interface.

3.22 The linear completion protocol

The clamp–relax–complete–regulate language can now be instantiated for global linear models.

Clamp

The visible feature coordinates are fixed. For supervised prediction, the target component is masked:

$$(x_*, ?).$$

Relax

The observed features are projected onto the learned task-relevant direction or directions:

$$s_* = x_*W + b.$$

Unlike k-NN, there is no iterative physical relaxation in the classical algorithm. The term here refers to propagation through the fitted global relation.

Complete

A task-specific readout produces the missing value:

$$\widehat{y}_* = s_*$$

for regression,

$$\widehat{p}_* = \sigma(s_*)$$

for binary logistic completion, or

$$\widehat{p}_{*,c} = \frac{e^{s_{*,c}}}{\sum_r e^{s_{*,r}}}$$

for multiclass softmax completion.

Regulate

Classical linear models do not have a single regulation stage, but several mechanisms already play regulatory roles:

- ridge or lasso constrains fitted directions;
- validation selects regularization strength;
- residual diagnostics test model adequacy;
- condition number diagnoses unstable feature geometry;
- leverage identifies geometrically influential samples;
- margin measures classification fragility.

The important conceptual point is that regulation appears in several places once we stop identifying the model solely with its final predictor.

3.23 Companion experiment: local versus linear completion

The companion project should now compare two fundamentally different uses of geometry on the same data and masks.

Let

$$\widetilde{Z} = [X \mid y]$$

be the augmented data matrix. Standardize numerical features using training-set statistics only.

Task A: regression completion

Choose a numerical target or construct a regression task. Compare:

1. global mean completion;
2. k-NN regression;
3. least squares;
4. ridge regression across a validation-selected grid of λ.

Report test error, coefficient norm, condition number, and the selected λ. Plot residuals against fitted values.

Task B: binary label completion

Compare:

1. class-frequency baseline;
2. k-NN classification;
3. logistic regression;
4. regularized logistic regression.

Report accuracy or balanced accuracy, log loss, margin distribution, and coefficient norm. Also record the k-NN radius and entropy diagnostics from section 2.9.

The aim is not merely to ask which method wins. The experiment should identify *why* each method succeeds or fails.

For every error, ask whether it is associated primarily with:

- sparse local support;
- local target disagreement;
- small linear margin;
- high leverage;
- systematic residual curvature;
- unstable or highly correlated feature directions.

This produces a richer comparison than a leaderboard. The two model families expose different failure geometries.

Task C: perturb the geometry

Repeat the comparison after one deliberate transformation:

1. rescale one feature by a large constant;
2. add irrelevant Gaussian-noise features;
3. add a polynomial or interaction feature known to be relevant.

Track how the local neighborhoods, linear coefficients, margins, and errors change.

This experiment makes the chapter's main claim visible: methods respond differently because they organize completion through different geometric structures.

3.24 A minimal software interface

A linear completer in the companion code can implement the same interface introduced for local completion:

```
fit(X_full, mask_pattern=None)
complete(X_partial, mask)
    -> X_completed, diagnostics
```

For a supervised target-column mask, diagnostics should include at least:

- fitted coefficients and intercept;
- coefficient norm;
- training and validation loss;
- regularization strength;
- condition number or singular-value summary;
- residual statistics for regression;
- margin or score statistics for classification.

The interface matters because it prevents us from treating each algorithm as a separate software world. The mask defines what is absent; the completer defines how geometry is used to recover it; diagnostics expose the conditions under which the result was obtained.

3.25 From fitted directions to latent directions

Linear supervised models learn directions because the target tells them what matters. But a data matrix also contains structure before any particular target is chosen.

Suppose many feature columns are strongly correlated. The observations may lie close to a lower-dimensional subspace:

$$x_i \approx U z_i, \qquad z_i \in \mathbb{R}^r, \quad r \ll d.$$

Instead of fitting one target direction w, we can ask for a small set of latent directions that reconstruct the entire data matrix.

The completion question then changes from

$$x \longmapsto y$$

to

$$X_{\text{observed}} \longmapsto \text{latent structure} \longmapsto \widehat{X}.$$

This leads to principal component analysis, singular-value decomposition, low-rank approximation, and matrix completion.

The conceptual transition is:

linear supervised completion	:	fit target-relevant global directions,
latent completion	:	discover directions that organize the data matrix itself.

The next chapter develops this second form of global geometry.

3.26 Summary

The main conclusions of this chapter are:

1. Linear completion replaces query-dependent neighborhoods by global fitted directions in feature space.
2. Least squares is an orthogonal projection of the target vector onto the column space of the design matrix.
3. The hat matrix is a model-induced hard mask: in a basis adapted to the fitted subspace it passes exactly the representable target-pattern directions and zeros their orthogonal complement; ridge replaces this by a soft spectral mask.
4. The feature Gram matrix $X^\top X$ determines how feature relations affect the fitted direction.
5. Multicollinearity and poor conditioning indicate directions that are weakly constrained by the observed data.
6. Ridge regularization stabilizes weak singular directions by changing the geometry of admissible parameter vectors.
7. Other penalties, such as L_1, encode different parameter geometries and can produce sparse completion rules.
8. Linear classification uses the same fitted-score structure as regression but changes the readout: threshold, logistic probability, softmax probability, or margin-based decision.
9. Margin, leverage, residual structure, and condition number are diagnostics distinct from the final prediction.
10. A linear rule can be powerful when the representation already makes the target simple; linear versus nonlinear is relative to representation.
11. Basis expansion extends linear completion by changing the representation, preparing the move toward learned representations.
12. Arbitrary masks reveal the need for joint or latent models rather than a single target-specific coefficient vector.
13. The next step is to learn global latent directions that reconstruct the data matrix itself rather than only a designated target.

Linear models are therefore not merely elementary predictors. They are the first compact machines in the book that *fit* a task-dependent global geometry. They compress a training

sample into directions, expose the relation between geometry and regularization, and make clear that successful completion depends as much on the representation and the support of fitted directions as on the final prediction rule.

Chapter 4

Latent Completion: PCA, SVD, and Low Rank

4.1 From fitted directions to latent structure

The previous chapter studied supervised linear models. There the representation of each sample was given by its feature vector x, the training data supplied feature–feature relations such as $X^\top X$, and the target was used to fit a task-specific relation

$$x \longmapsto \widehat{y}.$$

The fitted coefficient vector summarized how the observed features should act on that particular target.

This chapter removes the designated target. We ask whether the data themselves admit a simpler representation in which many observed coordinates are coupled by a smaller number of degrees of freedom. The central question is therefore

Can a lower-dimensional representation preserve enough structure to reconstruct what an observation leaves unresolved?

The basic pattern is

$$X \longrightarrow Z \longrightarrow \widehat{X},$$

where

$$Z \in \mathbb{R}^{n\times r}, \qquad r \ll d,$$

is a lower-dimensional representation of the rows of

$$X \in \mathbb{R}^{n\times d}.$$

This distinction matters. The latent coordinates Z are a *representation*; they are not by themselves a geometry. Once we specify how latent objects are related—for example by Euclidean distance, an inner product, or a sample–feature compatibility—the latent representation induces a geometry that can be used for reconstruction or completion.

Principal component analysis, singular-value decomposition, clustering, low-rank approximation, and matrix factorization all fit this broad pattern, although they do not solve exactly

the same problem. Some compress a fully observed object, some infer latent organization, and some estimate genuinely unobserved entries.

The previous chapter introduced a second way to read a projection. The least-squares hat matrix acts as a *model-induced mask*: in a basis adapted to the fitted subspace, it passes exactly the target-pattern directions the linear model can represent and sets the complementary directions to zero. PCA will produce the same algebraic structure in a different space. Its projection acts in feature space rather than in the sample-indexed target space of regression. This connection gives a useful thread through the chapter: latent compression can often be understood as learning which directions of an object are allowed to remain present.

Principle 4.1 (Latent completion) *A latent completion uses an inferred lower-dimensional representation to couple observed quantities and thereby reconstruct information that is missing, suppressed, corrupted, or otherwise unresolved.*

The word *latent* does not imply a hidden physical cause. It means only that the coordinates used by the model are not supplied directly as observed columns. They are inferred or constructed from regularities in the data.

4.2 Why reduce dimension?

Suppose a centered data matrix is approximated by

$$X \approx ZW^\top,$$

with

$$Z \in \mathbb{R}^{n\times r}, \qquad W \in \mathbb{R}^{d\times r}, \qquad r \ll \min(n,d).$$

The factorization uses approximately

$$nr + dr$$

factor entries rather than treating the

$$nd$$

matrix entries as unrelated degrees of freedom.

The gain is not compression by itself. The factorization couples coordinates that would otherwise be independent. If several observed entries constrain the same latent variables, those variables can in turn constrain entries that were not observed.

Low dimension is therefore an inductive bias: the modeller assumes that many visible quantities can be explained through a smaller collection of shared degrees of freedom.

Several familiar situations have this form.

- In sensor data, many channels may respond to a few common physical processes.
- In images, repeated spatial patterns create dependence among many pixel coordinates.
- In recommender systems, many user–item preferences may be organized by a smaller number of user and item factors.
- In biological measurements, many variables may co-vary along a few dominant modes.

- In text representations, a large coordinate space may contain strongly correlated semantic or stylistic variation.

The observation process determines which evidence is available. The latent model determines how that evidence is coupled.

Example 4.2 (One latent factor) *Suppose three centered features satisfy approximately*

$$x_1 \approx 2z, \qquad x_2 \approx -z, \qquad x_3 \approx 3z.$$

A row is therefore represented by one latent coordinate,

$$x \approx z \begin{bmatrix} 2 & -1 & 3 \end{bmatrix}.$$

If the only observed coordinate is

$$x_2 = -2,$$

then the latent relation implies

$$z \approx 2.$$

The unresolved coordinates are consequently reconstructed as

$$\widehat{x}_1 \approx 4, \qquad \widehat{x}_3 \approx 6.$$

The completion does not come from a neighbouring row or from a coefficient fitted specifically to x_1 or x_3. It comes from the shared latent representation that constrains all three coordinates simultaneously.

This is the first point in the book where the matrix is used not merely as a collection of examples but as evidence from which a common internal representation can be constructed.

4.3 Clustering as a discrete latent representation

Latent structure need not be continuous. A dataset may be summarized by a finite set of states, regimes, prototypes, or groups. This is the classical problem of clustering: infer organization among samples without being given external class labels.

Clustering is useful here because it separates three ideas that are easy to conflate:

sample representation + relation rule + latent assignment.

The observed vector x_i represents sample i. A metric or other relation rule determines how samples or prototypes are compared. The cluster assignment z_i is an inferred latent variable.

4.3.1 The k-means objective

Let

$$x_1, \dots, x_n \in \mathbb{R}^{1\times d}$$

be the observed samples. Introduce latent assignments

$$z_i \in \{1, \dots, K\}$$

and prototypes

$$\mu_1, \dots, \mu_K \in \mathbb{R}^{1\times d}.$$

The standard objective is

$$J(z, \mu) = \sum_{i=1}^{n} \|x_i - \mu_{z_i}\|_2^2 .$$

With the prototypes fixed, Lloyd's assignment step chooses

$$z_i \leftarrow \arg\min_{k\in\{1,\dots,K\}} \|x_i - \mu_k\|_2^2.$$

With the assignments fixed, the prototype update is

$$\mu_k \leftarrow \frac{1}{|C_k|} \sum_{i:z_i=k} x_i.$$

The algorithm therefore alternates between two ordinary conditional optimizations:

$$\boxed{\text{prototypes} \longrightarrow \text{assignments} \longrightarrow \text{prototypes} \longrightarrow \cdots}$$

until the objective no longer decreases.

The geometry is explicit. The objects are samples and prototypes; the admissible relation compares each sample with each prototype; and the relation descriptor is Euclidean distance

$$W(x_i, \mu_k) = \|x_i - \mu_k\|_2.$$

The assignments z_i are not the geometry itself. They are a discrete latent representation induced by that geometry and by the k-means objective.

Definition 4.3 (Discrete latent geometry) *A discrete latent geometry is a latent organization in which objects are represented through a finite collection of states, groups, prototypes, or regimes, together with relations that determine how observed objects are assigned to that organization.*

> **Do not force the completion vocabulary.** Calling cluster labels "missing values" adds little by itself. The useful connection is that clustering infers structure that was not supplied in the observation. When ordinary clustering language is clearer, we will use it.

4.3.2 The geometry determines what a cluster can be

The same represented point cloud can yield very different clusterings under different relation rules. Rescaling a coordinate changes the representation seen by Euclidean k-means. Replacing Euclidean distance by another metric changes the induced Voronoi regions. A kernel changes the representation in which proximity is measured. Density- or graph-based methods can define groups through connectivity rather than proximity to a single prototype.

Thus

clusters are not simply "in the data"; they are inferred relative to a representation, a relation rule, and a criterion.

The number of groups K, the feature scaling, the metric, and the family of admissible group shapes are modelling choices.

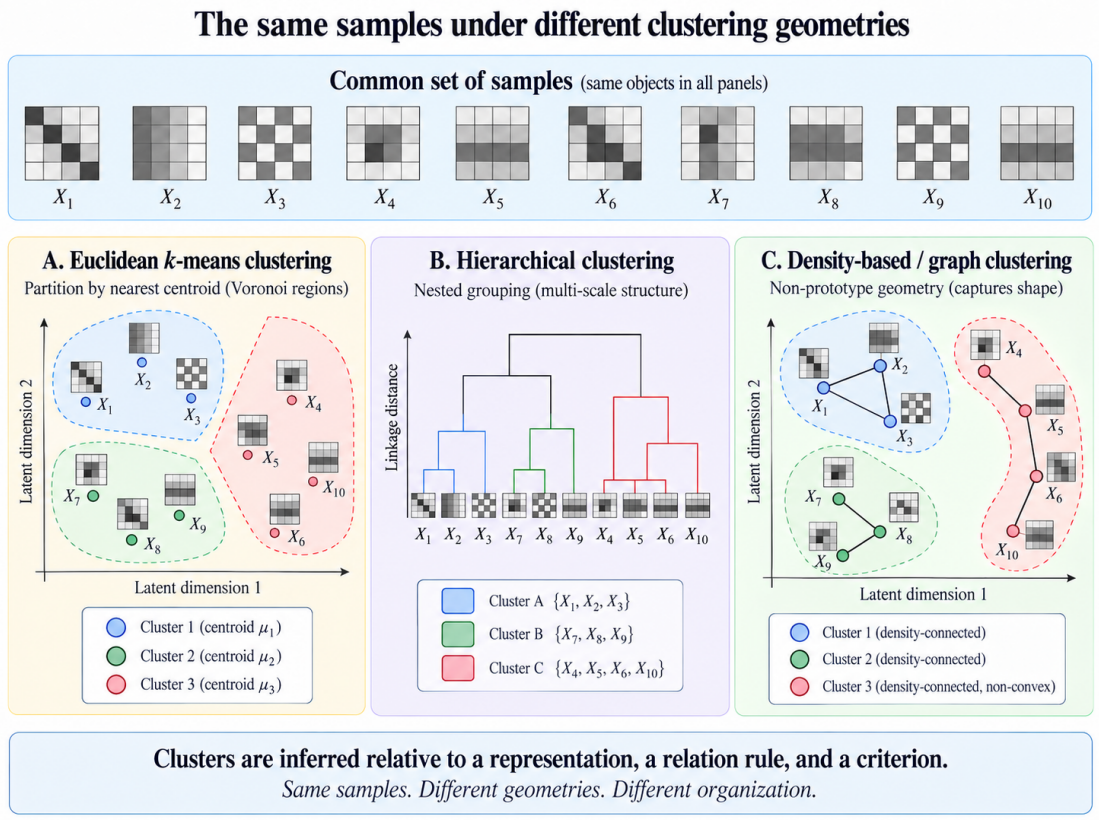


Figure 4.1: The same samples under different clustering geometries.

4.3.3 Hierarchies and soft memberships

Not every clustering method represents a group by a centroid. Hierarchical clustering constructs a nested family of relations or partitions across scales. Choosing a cut through the tree selects one resolution, but the hierarchy carries more information than that final cut.

Probabilistic mixture models replace hard assignments by responsibilities

$$r_{ik} = p(z_i = k \mid x_i),$$

so several latent groups may remain plausible for one sample. This creates a bridge to Chapter 7: ambiguity becomes part of the latent representation.

Different clustering families therefore encode different relational assumptions:

- k-means uses prototype-centred Euclidean regions;
- hierarchical methods preserve multiscale relations;
- mixture models retain uncertainty over latent states;
- density- and graph-based methods define groups through connectivity or density.

A broad survey is given by Jain (2010).

4.3.4 Clustering as a bridge from local to latent structure

Chapter 2 used neighbourhoods directly: nearby observed samples could provide evidence for a query. Clustering inserts an intermediate representation:

$$x_i \longrightarrow z_i \longrightarrow \mu_{z_i}.$$

The discrete code z_i discards within-cluster detail and preserves group identity. It also induces new relations, for example

$$R(i, j) = \mathbf{1}\{z_i = z_j\}.$$

PCA will shortly make an analogous move with continuous rather than discrete latent coordinates:

clustering	:	discrete latent representation,
PCA/factorization	:	continuous latent representation.

4.4 Centering, covariance, and the shape of a cloud

Let

$$X = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^{n \times d}, \qquad x_i \in \mathbb{R}^{1 \times d}.$$

For principal component analysis, we normally center each feature. Let

$$\mu = \frac{1}{n} \sum_{i=1}^{n} x_i \in \mathbb{R}^{1 \times d}$$

and define

$$X_c = X - \mathbf{1}\mu.$$

Centering is a change of representation:

$$\rho_c(x) = x - \mu.$$

The empirical covariance matrix is

$$S = \frac{1}{n-1} X_c^\top X_c.$$

Its entry

$$S_{rs}$$

describes how feature r and feature s co-vary across the observed samples. In the language of Chapter 1, S is therefore a constructed feature–feature relation descriptor.

If $v \in \mathbb{R}^{d\times 1}$ is a unit direction, the variance of the centered data projected onto v is

$$\mathrm{Var}(X_c v) = v^\top S v.$$

The first principal direction solves

$$v_1 = \arg\max_{\|v\|_2=1} v^\top S v,$$

and is the eigenvector of S associated with its largest eigenvalue. Subsequent principal directions maximize the same criterion subject to orthogonality.

We obtain orthonormal directions

$$v_1, \ldots, v_d$$

with

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0.$$

These directions define a new coordinate representation. With

$$V_r = [v_1, \ldots, v_r],$$

where the principal directions v_j are represented as column vectors. Since samples are row vectors, the r-dimensional representation of a sample is

$$z = (x - \mu)V_r.$$

If Euclidean distance is then used in this latent space, it induces the sample relation

$$d_r(x_i, x_j)^2 = \|(x_i - x_j)V_r\|_2^2.$$

This is the precise sense in which PCA can change the geometry: it first changes the representation, and the chosen relation rule in that representation induces different relations among the original samples.

Definition 4.4 (Principal component direction) *A principal component is an orthonormal direction in feature space chosen successively to maximize variance of the centered data subject to orthogonality to previously chosen directions.*

4.4.1 Variance is a geometric criterion, not a universal notion of importance

The first principal component is sometimes described as the "most important" direction. This is too strong. It is the direction of largest empirical variance under the chosen representation and inner product. A low-variance direction can still be decisive for a supervised target.

This distinguishes the present chapter from Chapter 3:

supervised linear direction	:	fitted for target relevance,
principal direction	:	constructed for reconstruction variance.

Whether the resulting representation is useful for a later prediction task must be tested rather than assumed.

4.4.2 Scaling changes the representation and the induced geometry

If one feature is measured in meters and another in millimeters, their variances can differ by orders of magnitude purely because of units. PCA on the covariance matrix will then privilege the high-variance coordinate.

Standardizing features to unit variance changes the representation before PCA is fitted. Performing PCA after standardization is equivalent to constructing principal directions from the correlation matrix rather than the covariance matrix. Since the representation has changed, so do the principal directions and any geometry subsequently induced in the latent space.

Principle 4.5 (Scaling is a modelling choice) *Because PCA is variance based, feature scaling changes the representation from which the latent coordinates are constructed. It must therefore be chosen for substantive reasons rather than treated as an administrative preprocessing detail.*

4.5 PCA as encoding and reconstruction

Let

$$V_r = \begin{bmatrix} v_1 & \cdots & v_r \end{bmatrix} \in \mathbb{R}^{d \times r}$$

contain the first r principal directions.

The latent coordinates of the centered data are

$$Z = X_c V_r \in \mathbb{R}^{n \times r}.$$

Each row z_i gives the coordinates of sample x_i in the retained principal subspace.

The reconstruction is

$$\widehat{X}_c = Z V_r^\top = X_c V_r V_r^\top.$$

Adding the mean back gives

$$\widehat{X} = \mathbf{1}\mu + X_c V_r V_r^\top.$$

For one sample,

$$x \xrightarrow{\text{encode}} z = (x - \mu) V_r \xrightarrow{\text{decode}} \widehat{x} = \mu + z V_r^\top.$$

Definition 4.6 (PCA reconstruction) *A PCA reconstruction is the orthogonal projection of a centered observation onto the subspace spanned by selected principal directions, followed by addition of the empirical mean.*

4.5.1 PCA projection as a latent mask

Define the rank-r projection

$$P_r = V_r V_r^\top.$$

Because the columns of V_r are orthonormal,

$$P_r^\top = P_r, \qquad P_r^2 = P_r.$$

Its complement

$$\overline{P}_r = I - P_r$$

is also an orthogonal projector, with

$$P_r \overline{P}_r = \overline{P}_r P_r = 0.$$

For a centered sample $x_c = x - \mu$,

$$x_c = \underbrace{x_c P_r}_{\text{retained by the latent model}} + \underbrace{x_c(I - P_r)}_{\text{suppressed by the latent model}}.$$

The PCA reconstruction is therefore

$$\widehat{x} = \mu + (x - \mu)P_r,$$

and its residual is

$$e = (x - \mu)(I - P_r).$$

This makes the connection with the hat matrix from Chapter 3 exact. Both H and P_r are symmetric idempotent operators. They differ in the space on which they act: H selects model-representable *target patterns across samples*, whereas P_r selects retained *feature directions within a sample*.

To see why P_r is literally a mask, extend V_r to an orthogonal basis

$$\widetilde{V} = \begin{bmatrix} V_r & V_\perp \end{bmatrix} \in \mathbb{R}^{d \times d}.$$

In this model-adapted basis,

$$\widetilde{V}^\top P_r \widetilde{V} = \begin{bmatrix} I_r & 0 \\ 0 & 0 \end{bmatrix}.$$

Thus PCA performs the binary operation

$$\begin{bmatrix} z_{\text{kept}} \\ z_{\text{discarded}} \end{bmatrix} \longmapsto \begin{bmatrix} z_{\text{kept}} \\ 0 \end{bmatrix}.$$

It is a hard mask, but not generally a mask in the original coordinate basis. It is a mask in a basis learned from the data.

At this point three different masks should be kept separate:

mask		space/basis	what it suppresses
coordinate mask	observation	original coordinates	unobserved entries
H		sample-indexed target-pattern space	patterns a supervised linear model cannot represent
P_r		principal feature basis	feature directions excluded by the rank-r model

The first mask is imposed by the observation process. The latter two are *model-induced masks*: they are consequences of what the fitted model is able, or chooses, to represent.

Principle 4.7 (Projection as a model-induced mask) *A projection is a hard mask in its adapted basis. It passes represented directions and suppresses their orthogonal complement. That basis need not be the original coordinate basis.*

4.5.2 PCA reconstruction is not entrywise missing-data completion

A standard PCA row may be fully observed. Nothing in the original vector needs to be hidden by an *observation* mask. Nevertheless the fitted PCA model itself contains the projection mask P_r: it deliberately suppresses directions outside the retained principal subspace. The information bottleneck occurs because the decoder receives only

$$z = (x - \mu)V_r,$$

not the full d-dimensional vector.

Thus the encoder itself acts as an observation operator:

$$\mathcal{O}_r(x) = (x - \mu)V_r.$$

The decoder reconstructs the original representation from this compressed observation,

$$z \longrightarrow \widehat{x}.$$

This is reconstruction from a *projected observation*, not ordinary missing-entry imputation. The model-induced mask and an external coordinate mask are different operators and may coexist in the same problem. If coordinates are genuinely missing, one cannot in general apply P_r to a zero-filled vector and call the result PCA completion; the latent coordinates must first be inferred from the information actually observed.

The part suppressed by the PCA mask is the component orthogonal to the retained subspace,

$$e = x - \widehat{x} = (x - \mu)(I - P_r),$$

with squared error

$$\|e\|_2^2.$$

Across the dataset,

$$\|X_c - \widehat{X}_c\|_F^2$$

measures the information discarded by rank-r compression.

This distinction will matter later. Standard PCA assumes complete input when its subspace is fitted. PCA or factor models with genuinely missing coordinates require an additional inference step.

4.6 Explained variance and selecting the latent dimension

The total variance of centered data is

$$\operatorname{tr}(S) = \sum_{j=1}^{d} \lambda_j.$$

The variance represented by the first r components is

$$\sum_{j=1}^{r} \lambda_j.$$

The explained variance ratio is therefore

$$\mathrm{EV}(r) = \frac{\sum_{j=1}^{r} \lambda_j}{\sum_{j=1}^{d} \lambda_j}.$$

The mask interpretation gives the same quantity an immediate energy meaning. Because P_r and $I - P_r$ are complementary orthogonal projectors,

$$\|X_c\|_F^2 = \|X_c P_r\|_F^2 + \|X_c(I - P_r)\|_F^2.$$

Hence

$$\mathrm{EV}(r) = \frac{\|X_c P_r\|_F^2}{\|X_c\|_F^2},$$

while

$$1 - \mathrm{EV}(r) = \frac{\|X_c(I - P_r)\|_F^2}{\|X_c\|_F^2}$$

is the fraction of centered data energy suppressed by the rank-r projection mask.

This provides a convenient summary of compression, but it should not be mistaken for an oracle that chooses the correct dimension. A threshold such as 90% or 95% explained variance is a convention, not a theorem.

The choice of r encodes a bias–variance and compression–fidelity tradeoff:

	small r	large r
compression	strong	weak
reconstruction bias	high	low
noise retained	less	more
model complexity	low	high

For a reconstruction or completion task, a stronger procedure is to select r by validation under the same kind of observation process expected at use time. For missing entries this means a representative validation mask; for denoising it means representative corruption. This keeps model selection aligned with the actual problem rather than with variance alone.

4.7 Singular-value decomposition

PCA is often introduced through the covariance matrix. The singular-value decomposition gives a direct factorization of the centered data matrix itself.

For a centered matrix X_c of rank q, write

$$X_c = U\Sigma V^\top,$$

where

$$U \in \mathbb{R}^{n\times q}, \qquad V \in \mathbb{R}^{d\times q},$$

have orthonormal columns and

$$\Sigma = \operatorname{diag}(\sigma_1, \dots, \sigma_q), \qquad \sigma_1 \geq \cdots \geq \sigma_q > 0.$$

The columns of V are feature-side modes. The columns of U are sample-side modes. The singular values measure the strength of the paired modes. The covariance matrix is

$$S = \frac{1}{n-1} V\Sigma^2 V^\top,$$

so

$$\lambda_j = \frac{\sigma_j^2}{n-1}.$$

The SVD is especially clear in the bipartite view of a matrix. Define latent coordinates

$$p_i = \begin{bmatrix} \sqrt{\sigma_1}u_{i1} & \cdots & \sqrt{\sigma_q}u_{iq} \end{bmatrix}^\top$$

for sample i, and

$$q_j = \begin{bmatrix} \sqrt{\sigma_1}v_{j1} & \cdots & \sqrt{\sigma_q}v_{jq} \end{bmatrix}^\top$$

for feature j. Then

$$(X_c)_{ij} = p_i^\top q_j.$$

Thus the SVD supplies latent representations for two object types—samples and features—while the inner product specifies how those representations relate. This is more precise than saying that U and V "are the geometry".

4.7.1 Outer-product decomposition

The SVD can also be written

$$X_c = \sum_{j=1}^{q} \sigma_j u_j v_j^\top.$$

Each rank-one term

$$\sigma_j u_j v_j^\top$$

links one pattern across samples to one pattern across features.

A rank-r approximation keeps only the strongest terms:

$$X_r = \sum_{j=1}^{r} \sigma_j u_j v_j^\top = U_r \Sigma_r V_r^\top.$$

For centered data, truncated SVD and PCA reconstruction are two views of the same low-rank representation.

Definition 4.8 (Rank-r approximation) *A low-rank approximation of a matrix is a matrix of rank at most r chosen to approximate the original matrix according to a specified loss, most commonly squared Frobenius norm.*

4.8 Why truncated SVD is the best low-rank reconstruction

Among all matrices of rank at most r, the truncated SVD solves

$$X_r = \arg \min_{\operatorname{rank}(Y)\le r} \|X_c - Y\|_F.$$

This is the classical Eckart–Young–Mirsky result.

Moreover,

$$\|X_c - X_r\|_F^2 = \sum_{j=r+1}^{q} \sigma_j^2.$$

The discarded singular values therefore quantify exactly the squared reconstruction error under the optimal rank constraint.

The important modelling assumption is the rank constraint. It says that a small number of shared sample–feature modes should account for most of the observed matrix. The latent representation then induces relations among entries that were not independent in the original table.

Principle 4.9 (Low-rank structural bias) *A low-rank model couples many matrix entries through a small collection of shared sample–feature modes. Reconstruction or completion is possible only to the extent that this shared structure is appropriate to the data and supported by the observation.*

4.8.1 A spectral filter

Truncated SVD makes the mask interpretation visible from both sides of the matrix. Define

$$P_r^{(\mathrm{s})} = U_r U_r^\top, \qquad P_r^{(\mathrm{f})} = V_r V_r^\top.$$

These are projection masks in sample-pattern space and feature-pattern space, respectively. The same rank-r reconstruction can be written

$$X_r = P_r^{(\mathrm{s})} X_c = X_c P_r^{(\mathrm{f})} = P_r^{(\mathrm{s})} X_c P_r^{(\mathrm{f})}.$$

The SVD couples the two masks through the same retained singular modes.

In singular coordinates the masking is especially simple. Truncation applies the binary spectral mask

$$\sigma_j \longmapsto \begin{cases} \sigma_j, & j \le r, \\ 0, & j > r. \end{cases}$$

Equivalently, if

$$D_r = \operatorname{diag}(\underbrace{1, \dots, 1}_{r}, 0, \dots, 0),$$

then

$$X_r = U\Sigma D_r V^\top.$$

This makes the connection with ridge regression in Chapter 3 sharper. Ordinary least squares and truncated SVD use hard 0/1 projection masks in model-adapted bases. Ridge uses a soft spectral mask, replacing a binary gain by

$$\frac{\sigma_j^2}{\sigma_j^2 + \lambda} \in (0, 1).$$

Thus hard projection and soft shrinkage are two ways of controlling which directions remain present to a linear model.

	ridge	truncated SVD
fitted object	target relation	data representation
spectral action	soft shrinkage	hard truncation
criterion	supervised loss + penalty	reconstruction loss
result	regularized predictor	low-rank reconstruction

4.8.2 From the SVD to a completion operator

The SVD is useful not only because it decomposes a matrix, but because it turns the decomposition into explicit operators that can be used for reconstruction and diagnosis.

Let the centered data matrix have compact SVD

$$X_c = U\Sigma V^\top,$$

and let

$$U_r, \qquad \Sigma_r, \qquad V_r$$

contain the first r singular modes. Define the two orthogonal projectors

$$P_r^{(s)} := U_r U_r^\top$$

and

$$P_r^{(f)} := V_r V_r^\top.$$

The superscripts remind us that the first projector acts on the *sample side*, while the second acts on the *feature side*.

The rank-r reconstruction can then be written in three equivalent ways:

$$\boxed{X_r = U_r \Sigma_r V_r^\top = P_r^{(s)} X_c = X_c P_r^{(f)}.}$$

This identity is worth understanding rather than merely memorizing. It says that the same low-rank reconstruction can be obtained by

retaining the first r singular modes,

projecting the rows through the retained sample subspace,

or projecting the columns through the retained feature subspace.

Thus the two-sided geometry exposed by the SVD is operational. The sample and feature descriptions are not two unrelated summaries of the matrix: they generate the same reconstructed relation.